

Hybrid-INoT: разделение функций по ролям и внешняя проверка при генерации кода

Даниил Привезенцев^{1*}

^{1*} Национальный исследовательский университет «Высшая школа экономики», Москва, Россия.

Для корреспонденции: daprivezentsev@edu.hse.ru;

Аннотация

Hybrid-INoT объединяет планирование, реализацию и критику внутри одного ответа модели с последующей проверкой программы тестами. Мы формализуем однократное ядро и условный баланс ресурсов, затем исследуем ядро без компрессии и повторных генераций по результатам проверки. Факторный дизайн сочетает нейтральные или ролевые обозначения этапов с одним или тремя вызовами при неизменных операциях и полном входном контексте; дополнительное сравнение даёт прямой решатель. На 200 зарезервированных задачах BigCodeBench модель GPT-5.4 mini с уровнем рассуждения medium даёт 996 завершённых программ из 1 000 назначений по протоколу с раскрытым изменением процедуры выполнения. Все полученные программы проверены штатными тестами; эталонные контроли пригодны для 193 задач. Доля успешных проверок составляет 44.56–48.70%. Разности между ролевыми и нейтральными условиями равны -1.05 процентного пункта для одного вызова и +1.04 для трёх, с описательными 95%-интервалами [-4.71, +2.62] и [-3.11, +5.70]. Ни один из четырёх заранее заданных тестов качества не отклоняет нулевую гипотезу после поправки Холма. На полных ресурсных парах оценка по тарифам API у одновызовного ролевого условия ниже на 18.45%; три вызова требуют в 2.09–2.50 раза большей оценки, чем соответствующие одновызовные условия. Отдельный эксперимент с обозначениями и оформлением на 80 задачах даёт 320 программ, проверенных штатными тестами; убедительного снижения ресурсов или улучшения качества не обнаружено. Все десять назначений демонстрации SWE-bench по одной задаче исправления ошибки оценены; ни одно не решает задачу. Контрольные проверки, полные таблицы исходов и явный учёт пропусков определяют область выводов о компонентах метода. Результаты не устанавливают ни неухудшение качества, ни общее преимущество внутренних ролей.

Ключевые слова: Hybrid-INoT, разделение функций по ролям, внешняя проверка, токенная эффективность, факторный дизайн, генерация кода

1 Введение

Система для разработки программ должна интерпретировать спецификацию, использовать относящиеся к задаче код и документацию, построить решение и проверить его тестами. Эти функции можно распределить между этапами планирования, реализации и проверки. Если отдельные вызовы модели получают одни и те же материалы, контекст приходится передавать повторно. Мы исследуем, можно ли сохранить эти функции при меньшем числе вызовов, оставив проверку корректности внешним тестам.

Hybrid-INoT отвечает на этот вопрос сочетанием нескольких предписанных функций в одном ответе модели и последующей проверки программы исполняемыми тестами. Планирование, реализация и проверка задаются в общем вызове модели, а внешняя процедура оценивает полученную программу. Дизайн развивает мотивацию внутреннего диалога INoT [5], но исследуемая ролевая конфигурация отличается от исходного алгоритма PromptCode. Она отделяет генерацию программы от проверки её корректности. Длинный общий контекст мотивирует модель затрат, но сам по себе не гарантирует сохранения качества при одном вызове.

Исходная реализация Hybrid-INoT включала также отбор контекста и избирательные повторы. Сравнение целых конфигураций изменяло эти механизмы вместе с топологией вызовов и потому не выделяет вклад внутренних ролей. Настоящая оценка изолирует одношаговое ядро архитектуры: полный предоставленный контекст, один итоговый кандидат, предписанные планирование, реализацию и проверку, а также последующее штатное тестирование. Компрессия и повторы по результатам проверки отключены. Раздельная оценка компонентов позволяет проверить проектные решения Hybrid-INoT в рамках исходной инженерной задачи.

У ядра есть два разных эмпирических вопроса. Меняют ли названия ролей результат при неизменных операциях этапов? Как выполнение этих операций в одном ответе соотносится по качеству и ресурсам с их распределением по трём вызовам? Прямой решатель проверяет полезность поэтапной организации в целом. Отдельная адаптация INoT исследует родственную алгоритмическую альтернативу. Каждое условие получает одинаковую задачу и контекст и оценивается одной последующей процедурой. Внутренняя проверка, описанная в запросе, не считается отдельным измеренным исходом.

Вклад работы включает:

1. Операционное описание одношагового ядра Hybrid-INoT, его состояния и границы внешней оценки с прямым соответствием каждому реализованному контролю.

2. Условный баланс токенов общего контекста, объясняющий возможность экономии при меньшем числе вызовов без предположения о равной длине выхода и без заявления измеренного порога длинного контекста.
3. Компонентное исследование 2×2 без компрессии и прямой решатель на 200 резервных случайных задачах BigCodeBench: 996 наблюдаемых программ, исходные тесты, четыре заранее заданных парных контраста качества и явный учёт пропусков.
4. Дополнительные опыты: 400 программ на этапе разработки, 199 программ адаптации INoT, ограниченная штатная проба репозиторных патчей и полные таблицы исходов в приложении.

Абляции ролей и исследования числа взаимодействий имеют близких предшественников; вклад заключается в определённой архитектурной оценке и её проверяемых данных, а не в изобретении ролевого сотрудничества. Основные результаты ограничивают проектный вывод: ролевые обозначения в одном вызове сопровождаются меньшим наблюдаемым средним расходом, но сохранение качества не установлено; три вызова требуют больше ресурсов без обнаруженного улучшения качества в четырёх запланированных тестах.

2 Связанные исследования

2.1 Ближайший методологический предшественник

В работе Self-collaboration Code Generation via ChatGPT [1] уже исследованы ролевые инструкции и максимум взаимодействий в разделах 5.2–5.3. Поэтому удаление ролей или изменение взаимодействий не заявляется новизной настоящего исследования. Таблица 1 разграничивает реализованные воздействия.

Таблица 1 Сопоставление с ближайшим предшественником [1], таблицы 3–4 и приложение B.1. Раунды взаимодействия и новые вызовы модели являются разными величинами.

Аспект	Статья Self-collaboration	Компонентный эксперимент
Обозначения	Ролевые инструкции и инструкции без ролей	Одинаковые предложения об операциях; меняются только префиксы этапов
Взаимодействие	Цикл обратной связи; варьируется максимум взаимодействий	Один или три новых вызова; фиксированная последовательность, полная видимая история
Дизайн	Раздельные абляции ролей и взаимодействия	Совместная матрица обозначений и вызовов для каждой отобранной задачи
Задачи	HumanEval, MBPP и расширенные тесты	Резервная выборка BigCodeBench; эталонные и неправильные контроли до генерации

Добавленная ценность состоит в репликации и расширении с акцентом на проверяемость: совместный дизайн оценивает взаимодействие обозначений и топологии при одном интерфейсе генерации, а связь результатов штатных тестов с проверенными программами и компоненты токенов делают различия качества и ресурсов проверяемыми. Это новые эмпирические условия и набор доказательств, а не новый принцип ролевого сотрудничества или первая абляция ролей.

Self-collaboration является ближайшим внешним методом: его абляция ролей затрагивает тот же вопрос об источнике эффекта. Во внешнем сравнении используется закреплённая авторская реализация 2024 года, которая исполняет сгенерированные тесты, тогда как статья 2023 года описывает имитацию тестирования (раздел 13.3). Зафиксированное сравнение назначает SCC, новые SR и SN на те же 1 000 задач с тремя повторами; варианта SCC без ролей в нём нет. Поэтому оно оценивает различия полных процедур, включая обратную связь и остановку, а SR–SN и MR–MN выделяют эффект заданных обозначений в компонентном дизайне. Две проверки разработки подтверждают работоспособность процедуры, но основная серия ещё не завершена. Д служит минимальным сравнением. INoT* отвечает на отдельный вопрос об алгоритме внутреннего диалога.

2.2 Внутренний диалог и репозиторные методы

INoT описывает компактные инструкции PromptCode для внутреннего диалога и рассуждения [5]. Это мотивирует оценку раскрытой адаптации алгоритма, тогда как отдельное минимальное воздействие обозначений проверяет, меняют ли результат сами названия ролей. Сравнения отвечают на разные вопросы: инструкция INoT изменяет процедуру рассуждения, а факторный контраст обозначений сохраняет предложения с описанием операций. Исходное сравнение Hybrid-INoT дополнительно изменяло компрессию и повторные попытки; его прежние условия рассматриваются только как контекст и не объединяются с новым экспериментом.

Исследование одно- и многоагентных систем при сопоставимом бюджете токенов рассуждения подчёркивает различие дополнительных вызовов и вычислений [6]. Поскольку здесь длина завершения не ограничена общим жёстким бюджетом, токены и денежная оценка рассматриваются вместе с качеством как исходы реализованных протоколов; оптимальное распределение фиксированного бюджета инференса из этого дизайна не следует.

Agentless использует структурированный процесс локализации, исправления и проверки [7]; SWE-agent связывает модель с интерфейсом репозитория [8]. Обе системы добавляют поиск, инструменты и взаимодействие с окружением сверх генерации функций при фиксированном контексте. Их опубликованные оценки нельзя вставлять в парную матрицу с другими задачами и настройками. BigCodeBench [9] предоставляет разнообразные задачи с библиотечными зависимостями и исполнимыми тестами, а SWE-bench [10, 11] проверяет изменения репозитория тестами конкретных задач из трека ошибок. Штатная оценка связывает исходы с исполняемыми тестами, однако успешный эталонный контроль не разрешает каждое расхождение естественно-языковой инструкции и теста.

Обратная связь является отдельным выбором в дизайне метода. Self-Refine чередует замечания и исправления, полученные от той же модели [3]. Reflexion сохраняет словесные выводы между попытками; в его экспериментах с программированием обратную связь дают исполняемые тесты, созданные моделью [4]. Условия нашего компонентного эксперимента не получают результатов исполнения во время генерации и следуют фиксированной последовательности, поэтому не воспроизводят эти адаптивные процедуры. Авторская реализация SCC использует сгенерированные тесты как часть своей полной процедуры. В наших экспериментах тесты бенчмарка и эталонные программы недоступны интерфейсу генерации; внутреннее исполнение SCC не определяет успех на бенчмарке. Поэтому доступ к обратной связи и остановка входят в сравнение внешних методов, а не доказывают эффект ролевых обозначений. Self-Refine и Reflexion включены в методологическое сопоставление; экспериментальные запуски этих методов здесь не выполнены.

Исследования persona prompting мотивируют более узкое утверждение, чем универсальная польза ролей для рассуждения. Zheng и соавторы не обнаруживают устойчивого улучшения от системных персон в исследованных фактологических задачах [16]. Luz de Araujo и соавторы разделяют качество, устойчивость к несущественным атрибутам персоны и соответствие заданной специализации; результаты зависят от модели и задачи [17]. Эти работы не проверяют наши запросы на генерацию кода. Они обосновывают изменение обозначений при сохранении требуемых операций и ограничение вывода конкретными использованными обозначениями.

Данные о нескольких агентах также зависят от выбранного сравнения. Choi и соавторы связывают значительную часть выигрыша в своих NLP-экспериментах с независимым голосованием, отделяя обмен сообщениями от агрегации [18]. В то же время Wunderlich и соавторы находят конфигурации, в которых debate или mixture-of-agents улучшают соотношение вычислений и точности относительно self-consistency на MMLU-Pro и ВВН [19]. Это основание для условной оценки, а не универсального вывода за или против нескольких агентов. Наша нейтральная последовательная цепочка не является голосованием, смесью независимых моделей или ансамблем с выровненным бюджетом. В работе выполнена факторная проверка одной модели на исполняемом коде с сохранением трасс и учётом доступных счётчиков ресурсов; общее превосходство над перечисленными альтернативами не входит в оцениваемый эффект.

3 Формальная постановка и ядро Hybrid-INoT

3.1 Инженерная задача и наблюдаемый успех

Инженерная задача задаётся тройкой $(x, \mathcal{C}_0, \mathcal{V})$: спецификацией x , фиксированным предоставленным контекстом $\mathcal{C}_0$ и внешней процедурой проверки $\mathcal{V}$. Результат включает программу-кандидат y и запись её проверки. Генерация и проверка являются разными операциями: модель предлагает решение, а исходные исполняемые тесты определяют исход бенчмарка в контролируемом окружении.

Текстовое утверждение о корректности не заменяет тесты. Для задач с репозиторием C_0 обозначает раскрытый пакет найденных файлов, а не предположение о передаче всего репозитория.

Для протокола исполнения a инженерная цель состоит в повышении вероятности успешного прохождения тестов при контроле ожидаемого числа токенов и денежной оценки генерации. Поэтому качество и ресурсы сообщаются совместно, без оптимизации непроверенного взвешенного показателя. Наблюдаемые программы, провалившие тесты, считаются неуспешными; недоступную генерацию или оценку нельзя превращать в успешный исход либо в вымышленный отказ тестов. Раздел 5 задаёт общий контроль оцениваемости и точный статистический исход.

3.2 Разделение функций и внешняя проверка

Hybrid-INoT задаёт в ответе модели три функции: планирование, реализацию и проверку требований; сама проверка программы выполняется после генерации. «Гибридность» означает сочетание инструкций в запросе с последующими исполняемыми тестами. Для этого не требуется обученный маршрутизатор. Исходная терминология `planner-worker-critic` соответствует исполняемым обозначениям `planner`, `implementer` и `reviewer`. Здесь изучается разделение предписанных функций, а не наличие независимо наблюдаемых скрытых агентов.

Исследуемое одновызовное ядро предписывает три операции: анализ требований и граничных случаев, реализацию по плану, затем проверку и исправление относительно требований. Один вызов модели получает все три инструкции и полный предоставленный контекст; он возвращает одну итоговую программу. Отдельные внутренние планы и оценки не наблюдаются. Поэтому концептуальное разделение функций не следует принимать за три измеренные подпрограммы. Точные запросы, включая требование к итоговому формату, приведены в приложении.

В основном эксперименте SR представляет эту ролевую одновызовную конфигурацию. SN является её аблацией обозначений ролей при одинаковых предложениях об операциях. MR и MN распределяют соответствующие операции по трём новым вызовам, а D полностью убирает поэтапную процедуру. Для всех пяти условий используется одинаковая последующая штатная оценка. Дизайн позволяет оценить, как заданные этапы соотносятся с результатом по сравнению с простой инструкцией и что меняется при распределении тех же операций по отдельным вызовам. У Hybrid-INoT нет преимущественного доступа к проверке.



Рис. 1 Два варианта генерации: SR задаёт три функции одним запросом, MR распределяет их по трём вызовам. Стрелки между вызовами MR означают передачу полных предыдущих ответов; задача и контекст доступны в каждом вызове. Показан путь полученного ответа. Незавершённая генерация учитывается отдельно по алгоритму 1. Все пять условий из табл. 2 используют одну процедуру внешней оценки, без передачи результатов тестов обратно модели.

3.3 Состояние, исполнение и остановка

Наблюдаемое состояние исполнения задаётся как $\sigma_j = (x, \mathcal{C}_0, a, j, \mathcal{H}_j)$, где a фиксирует назначенное условие, j — номер вызова, а $\mathcal{H}_j$ содержит все полные предыдущие выведенные ответы. Конфигурация модели внутри сравнения фиксирована. Для D, SR и SN число вызовов $n_a = 1$; для MR и MN $n_a = 3$. В многовызовном условии $\mathcal{H}_j$ передаётся целиком. Предоставленный контекст не изменяется: $\hat{\mathcal{C}} = \mathcal{C}_0$. Условие назначается до генерации и потому не является политикой маршрутизации по результату.

Алгоритм 1 Генерация программы и проверка результата

Вход: задача, полный предоставленный контекст и заранее назначенное условие.

Выход: полученная программа, исход проверки и известный расход ресурсов.

I. Генерация

1. Определить число вызовов: один для D, SN или SR; три для MN или MR.
2. Выполнить вызовы последовательно. В каждый передать задачу, неизменный контекст и заданную инструкцию. Во второй и третий вызовы добавить все полные предыдущие ответы.
3. Для каждого вызова сохранить запрос, полный ответ, конечный статус и счётчики токенов.
4. Если вызов не завершён или учёт ресурсов не подтверждён, сохранить доступные сведения и остановить это назначение как недоступное. Не повторять его автоматически.
5. Из последнего завершённого ответа извлечь одну программу. Если формат неверен, сохранить пустого кандидата.

II. Проверка после окончания генерации

6. Проверить каждого полученного кандидата исходными тестами бенчмарка.
 7. Сопоставить проверенную программу, отчёт оценщика и результаты общих контрольных проверок.
 8. Записать успех, неуспех или недоступный исход. Сохранить все известные показатели ресурсов; не возвращать результаты тестов модели.
-

Алгоритм 1 описывает этапы генерации и оценки эксперимента; штатный оценщик запускается после генерации и не является инструментом внутри вызова модели. Лимит времени и конечное n_a ограничивают попытки генерации. Это правило остановки, а не теорема о сходимости или корректности. Отклонённое завершение не вызывает автоматическую повторную попытку той же ячейки. Раскрытое продолжение касается только назначений, в которых ни один вызов не начинался.

Конфигурация фиксирует дополнительные механизмы исходной архитектуры: отбор контекста тождественный, режим назначен, итоговый кандидат один, повторные генерации по проверке отключены. Так вопрос о заданных функциях

можно исследовать, не смешивая его с компрессией, малой проверяющей моделью и адаптивными повторами. Этим же определяется объём вывода: эксперимент оценивает данное ядро, а не все возможные варианты применения Hybrid-INoT.

4 Аналитическая модель затрат на общий контекст

4.1 Условный баланс токенов

Архитектурная мотивация связана с повторной передачей общего материала. Рассмотрим аддитивную модель учёта, в которой общий блок задачи и контекста вносит S токенов при каждой передаче. Внутренняя конфигурация H передаёт его один раз; внешняя M — в каждом из m вызовов. Пусть J_H, J_M содержат все остальные входные токены, включая оболочки инструкций и передаваемые ответы, а O_H, O_M — все выходные токены. Обозначим $R_H = J_H + O_H$, $R_M = J_M + O_M$. Тогда

$$T_H = S + R_H, \quad T_M = mS + R_M, \quad T_M - T_H = (m - 1)S + R_M - R_H. \quad (1)$$

Это тождество учёта при указанной модели блоков. Оно не предполагает равной длины выхода. В частности, более длинное рассуждение внутри одного вызова может компенсировать сокращение передачи контекста; внешние промежуточные ответы учитываются и при генерации, и при повторной передаче.

Утверждение 1 (Условное достаточное ресурсное преимущество). *Для $m > 1$ предположим, что $\mathbb{E}[R_H - R_M \mid S] \leq K$ во всём заданном диапазоне контекста, причём постоянная $K \geq 0$ не зависит от S в этом диапазоне. Из (1) следует*

$$\mathbb{E}[T_M - T_H \mid S] \geq (m - 1)S - K.$$

Поэтому ожидаемое преимущество внутренней конфигурации по токенам положительно при $S > K/(m - 1)$ внутри данного диапазона.

Доказательство Вычтем два баланса в (1), возьмём условное ожидание при S и применим предполагаемую верхнюю границу для $\mathbb{E}[R_H - R_M \mid S]$. $\square$

Ограничение остатка является содержательным предположением, а не измеренным свойством модели. Если выход или координационные затраты меняются с контекстом так, что граница нарушается, существование порога не следует. Условие сохраняет мотивацию Hybrid-INoT для длинного общего контекста и одновременно обозначает её пределы. В текущей выборке длина контекста не варьируется как воздействие и K не оценивается; определить порог преимущества или маршрутизации по ней нельзя. Прежние пилоты с компрессией также не определяют эти величины.

Эмпирический учёт использует фактические счётчики поставщика,

$$T_a = \sum_{j=1}^{n_a} (I_{a,j} + O_{a,j}), \quad (2)$$

а не оценивает суммы по (1). Незвестный расход остаётся неизвестным. Концептуальный блок S не отождествляется с измеренным аддитивным разбиением входа поставщика: границы токенизации и накладные расходы клиента могут препятствовать такому разбиению. Поэтому теоретический баланс не следует выдавать за наблюдаемую декомпозицию. При фиксированной топологии общий член сокращается между ролевым и нейтральным условиями; их ресурсная разность относится к изменениям формулировок и другим остаточным различиям. Поэтому в дизайне предусмотрены оба контроля обозначений.

4.2 Качество и деньги как отдельные ограничения

Обозначим успешное прохождение тестов через $Q_a = \mathbb{E}[Y_t(a)]$, а ожидаемое число токенов через $\bar{T}_a = \mathbb{E}[T_a]$. Исходное инженерное понятие токеной полезности можно записать как $U_a = Q_a/\bar{T}_a$, используя отношение ожиданий на общей полезностью наблюдаемой совокупности. Более высокое U_a может сочетаться с худшим Q_a и не подтверждает сохранение качества. Поскольку в исследовании различаются также доступные подвыборки качества и ресурсов, основной отчёт сохраняет отдельно парные разности качества и ресурсные отношения, не строя составной показатель полезности и не назначая непроверенных весов сопровождаемости.

Денежный аналог должен учитывать тариф каждого компонента: входа, кешированного входа и выхода. Оценки V и V_0 по API-тарифам в разделе 5 реализуют это требование по сохранённым счётчикам. Преимущество по токенам само по себе не устанавливает денежного преимущества при различающихся долях кеша и выхода. Ни одна из оценок не измеряет платёж по подписке или полную стоимость владения; вычисления оценщика, инженерная работа и эксплуатация лежат вне наблюдаемого учёта.

Баланс задаёт возможное объяснение ресурсного различия. Изменяется ли качество, полезны ли обозначения ролей и связана ли наблюдаемая ресурсная разность с конфигурацией одного вызова — эмпирические вопросы факторного эксперимента. Аналитическое объяснение не добавляет подтверждающей гипотезы к четырём тестам, зафиксированным до основного запуска.

5 Оценка компонентов: дизайн и оцениваемые эффекты

5.1 Факторный эксперимент без компрессии

Основной эксперимент оценивает ядро Hybrid-INoT и его контроли: независимо варьируются число вызовов модели (один или три) и обозначения этапов (нейтральные или ролевые). Таблица 2 определяет четыре факторных условия

и прямой решатель. Во всех структурированных условиях предписаны одинаковые операции: анализ требований и граничных случаев, реализация по плану, затем проверка и исправление относительно требований. Ролевое воздействие заменяет обозначения «stage 1», «stage 2», «stage 3» на «planner», «implementer», «reviewer»; предложения с описанием операций остаются неизменными. При одном вызове все три инструкции предшествуют задаче. При трёх вызовах каждый получает инструкцию своего этапа, полный текст задачи, весь предоставленный контекст и все полные предыдущие видимые ответы. Итоговый формат одинаков: ровно один блок с полной программой Python. Прямой решатель получает только инструкцию реализации и требование к итоговому формату.

Таблица 2 Реализованные условия. Фактор числа вызовов включает передачу промежуточных ответов между новыми вызовами.

Код	Вызовы	Обозначения	Операции
D	1	Нет	Прямая реализация
SN	1	Нейтральные	План, реализация, проверка
SR	1	Ролевые	План, реализация, проверка
MN	3	Нейтральные	План, реализация, проверка
MR	3	Ролевые	План, реализация, проверка

Компрессия отсутствует во всех условиях. Такая постановка заменяет прежнее сравнение трёх вызовов без сжатия с одним сжатым вызовом, но не оценивает эффект прежнего компрессора. Контраст обозначений является воздействием на инструкцию, а не свидетельством независимых внутренних агентов. Контраст числа вызовов включает поток промежуточной информации и повторный инференс, а не только механическое изменение счётчика вызовов. Непроверенные маршрутизатор и дополнительная модель проверки исключены из заявленного метода.

Пусть $Y_t(a)$ показывает, прошла ли единственная итоговая программа для задачи t в условии a исходные тесты бенчмарка в проверенном окружении. Четыре заранее заданных парных эффекта качества имеют вид

$$\Delta_{R,1} = \mathbb{E}_t[Y_t(\text{SR}) - Y_t(\text{SN})], \quad \Delta_{R,3} = \mathbb{E}_t[Y_t(\text{MR}) - Y_t(\text{MN})], \quad (3)$$

$$\Delta_{C,N} = \mathbb{E}_t[Y_t(\text{MN}) - Y_t(\text{SN})], \quad \Delta_{C,R} = \mathbb{E}_t[Y_t(\text{MR}) - Y_t(\text{SR})]. \quad (4)$$

Каждая нулевая гипотеза задаёт нулевую разность качества. Описательное взаимодействие равно $\Delta_{R,3} - \Delta_{R,1}$. Ожидания относятся к выборке задач при реализованном протоколе и стохастической генерации, а не к свойству, независимому от поставщика модели. Оценки по доступным данным требуют полных пар; при пропусках без дополнительных предположений они характеризуют именно наблюдаемую подвыборку.

5.2 Распределение задач и отдельный этап разработки

Источник задач — BigCodeBench v0.1.4 [9]. Основная выборка содержит 200 новых идентификаторов из случайной перестановки резервного пула: идентификаторы сначала сортировались лексикографически, затем перемешивались генератором Python с начальным состоянием 20260908. Взяты первые 200 допустимых идентификаторов в этом случайном порядке после исключения восьми примеров карточки набора данных (/0–/7), случайно просмотренных при проверке лицензии. Это не первые строки датасета и не отбор по сложности. Идентификаторы, входы модели и данные оценщика фиксируются до контрольных запусков; неудачный контроль не приводит к замене задачи. Сорок задач разработки не пересекаются с этими 200. Этап разработки включает два обозначенных повтора и 400 программ, основная выборка — один новый повтор и 1 000 назначений. Последние дают не более 200 независимых единиц выборки, а не 1 000 независимых задач. Родственные задачи могут дополнительно уменьшать эффективную независимость. Публичность бенчмарка оставляет открытым вопрос о наличии задач в обучающих данных.

Назначение означает запланированную генерацию для заданной задачи, условия и повтора; начатое назначение считается попыткой. Каждой основной задаче назначены все пять условий. Четыре непересекающиеся группы по остатку индекса от деления на четыре содержат по 50 задач. В каждой группе зафиксированный псевдослучайный порядок определяет блоки задач и порядок условий; группу обрабатывают два работника, одновременно действует не более восьми экспериментальных процессов CLI. Матрица назначений и реализация зафиксированы до отправки запросов. Построение входа и настройки оценщика приведены в приложении В. Модель не получает тесты бенчмарка, эталонные решения или результаты тестирования во время генерации. Метки повторов обозначают новые наблюдения по протоколу, а не случайные состояния модели у поставщика.

5.3 Генерация и полные доступные трассы

Используется модель с идентификатором `gpt-5.4-mini`, уровень рассуждения `medium` и официальный Codex CLI 0.153.4 с аутентификацией подписки ChatGPT [12, 15]. Опубликованный тариф позволяет проверить денежную оценку ресурсов; выбранная конфигурация является экономичным доступным вариантом для кода, но глобальный минимум цены не доказан. CLI работает с чистой конфигурацией без пользовательских инструкций и инструкций проекта. Инструменты, браузер, делегирование и выполнение тестов отключены. Каждый вызов создаёт новую временную сессию в пустом рабочем каталоге. Допустимая схема событий требует ровно одного полного ответа и корректной записи использования; неожиданный инструмент, повтор или событие сжатия нарушают заданное текстовое условие.

Для каждого отправленного вызова сохраняются точный промпт, аргументы, поток событий, стандартный поток ошибок и конечное состояние. Для завершённых записей дополнительно сохраняются итоговый текст и компоненты числа токенов. Для каждого последующего вызова проверяется, что его вход содержит

все предыдущие доступные ответы целиком. Доступность скрытых рассуждений поставщика не предполагается. Псевдоним модели не фиксирует неизменные веса; проверенное управление температурой и случайным состоянием модели отсутствует. На вызов установлен предел 180 секунд. Вход свыше 65 536 байт UTF-8 отклоняется, а не сокращается. Единого жёсткого ограничения выходных токенов нет; длина ответа и потребление ресурсов являются результатами воздействия.

Исходное правило останавливает группу после первой неудачной попытки. Поэтому сетевые обрывы и превышения времени оставляют другие назначения неотправленными. Отдельная поправка к протоколу, записанная до оценки или просмотра качества основной выборки, разрешает продолжить только такие неотправленные назначения. Любое назначение хотя бы с одним начатым этапом навсегда исключается из продолжения. Список оставшихся назначений и записи первой серии фиксируются до продолжения; сохраняются промпты, настройки, ограничения времени и исходный порядок внутри группы. Явный сетевой обрыв или превышение времени завершает попытку без повтора; ошибки квоты, аутентификации и нераспознанные ошибки останавливают дальнейшую отправку. Первые попытки, продолжения и пропуски остаются различимыми. Таким образом, анализ использует заранее заданные контрасты при изменённом протоколе исполнения; это не неизменённый пререгистрированный запуск.

5.4 Штатная оценка и контроль оцениваемости

Неизменённый оценщик BigCodeBench выполняет тесты набора v0.1.4 в режиме `instruct full`. Каждый запуск оценщика использует контейнер без сети, от непривилегированного пользователя, с основной файловой системой только для чтения, пределом памяти 3 ГБ, двумя CPU и двумя работниками оценщика. Настройки оценщика и зависимости, добавленные на этапе проверки эталонов, приведены в приложении В. Строгое извлечение выбирает единственный итоговый блок Python; нарушение формата даёт пустую наблюдаемую программу и отдельный признак ошибки формата, без выбора альтернативного ответа или исправления.

До генерации модели исходные эталонные и заведомо неверные программы проверены на всех 200 задачах. Изменения зависимостей ограничены этапом контролей, сохраняют предыдущие неудачные запуски и вызывают повторные контроли всей матрицы. Итоговое окружение пропускает 193 эталона и отклоняет все 200 неверных программ. Поэтому семь задач недоступны для оценки качества во всех условиях; их назначения и ресурсы генерации сохраняются в учёте. Итоговый образ дополняет окружение разработки фиксированными зависимостями и автономными ресурсами NLTK. Штатная диагностика эталонов содержит ошибки DNS (/101, /590), отсутствие TensorFlow (/418), неподдерживаемый аргумент кодировщика (/686), проверки моментов вырожденной выборки (/276) и проверки архива/журнала (/14, /1028). Последующая проверка без изменения образа подтверждает отсутствие внешних команд, используемых последними двумя задачами; связь этих отказов с отсутствием команд является диагностическим выводом по исходникам. Позднейшая диагностика не меняет зафиксированный

контроль оцениваемости. Контроли устанавливают оцениваемость и различие заведомо неверного решения, но не полную семантическую согласованность инструкции и теста.

Оценщику передаются только реально полученные программы. Каждый результат связывается с фактически проверенной программой, исходным идентификатором задачи и проверенным окружением; незавершённые запуски оценщика и противоречивые отчёты не могут давать наблюдения качества. Исходный штатный статус хранится рядом с отдельным аналитическим статусом с учётом контроля оцениваемости. Для отсутствующей генерации не создаётся фиктивный отказ тестов. Одинаковый контроль применяется ко всем условиям и поисковой репликации INoT.

5.5 Статистический анализ, пропуски и оценка ресурсов

Для каждого из четырёх запланированных контрастов качества приводятся число полных пар, оба числа несогласованных исходов, средняя парная разность, точный двусторонний биномиальный тест Мак-Немара и скорректированное по Холму p для семейства четырёх тестов при $\alpha = 0.05$. Только эти четыре теста Мак-Немара определяют основное семейство проверок; бутстрэп-интервалы и тесты смены знаков не служат критерием основного отклонения гипотезы. Процентильные интервалы используют 10 000 бутстрэп-выборок задач с начальным состоянием анализа 20260908; эти описательные интервалы не являются одновременными интервалами. Взаимодействие, ресурсные контрасты и сравнения с прямым решателем остаются описательными. Взаимодействие использует задачи с полными наблюдениями во всех четырёх факторных условиях; оно может не совпадать с вычитанием двух отдельно оценённых ролевых контрастов, если их доступные подвыборки различаются. Повторы разработки усредняются внутри задачи в отдельном описательном анализе и не объединяются с основными тестами.

При 200 независимых оцениваемых задачах наихудшая нормальная 95%-полуширина для одной доли приблизительно равна 6.9 процентного пункта. Это обоснование точности, а не расчёт 80%-мощности для парного эффекта; неопределённость парной разности зависит от несогласованных исходов. Приемлемая потеря качества для конкретного применения не получила внешнего обоснования, поэтому граница неухудшения и решение о сохранении качества не используются. Порог, выбранный только под статистическую точность, не устанавливал бы практически приемлемую потерю. Качество условия оценивается по доступным наблюдениям и сопровождается границами пропусков по всем назначениям: недоступные исходы считаются сначала всеми неудачами, затем всеми успехами. Эти границы не являются доверительными интервалами или поправкой при случайных пропусках.

Завершённый вызов CLI возвращает счётчики входа I , кешированного входа C и выхода O . Кешированный вход входит в общий вход; отдельно доступное число токенов рассуждения входит в выход. Повторного суммирования нет. Опубликованные стандартные тарифы API составляют соответственно USD 0.75, 0.075 и

4.50 за миллион токенов [14]. Рассчитываются

$$V = \frac{0.75(I - C) + 0.075C + 4.50O}{10^6}, \quad V_0 = \frac{0.75I + 4.50O}{10^6} \quad \text{USD.} \quad (5)$$

V — эквивалентная стоимость фактически использованных токенов по тарифу API, а не платёж по подписке; V_0 — анализ чувствительности без скидки за кеш. Обе оценки исключают работу исследовательских ассистентов, разработку кода, сборку окружения и вычисления оценщика. Расход на программу рассчитывается как сумма её завершённых вызовов; затем эти значения усредняются по программам. Отдельный реестр включает доступные счётчики прерванных программ и явно считает вызовы без итогового учёта; неизвестное использование не заменяется нулём. Отношения ресурсов используют одну и ту же подвыборку полных пар в числителе и знаменателе. «Меньшая наблюдаемая стоимость» означает отношение парных средних оценок ниже единицы, то есть отрицательную среднюю парную разность. Это описательный критерий: подтверждающий денежный порог и совместное правило решения о качестве и стоимости заранее не задавались; ресурсные интервалы не устанавливают денежное превосходство. Время отправки и состояние кеша могут влиять на денежную оценку, особенно между отдельно запущенными сериями.

6 Поисковая репликация алгоритма INoT

Четыре факторных условия проверяют отдельные компоненты, но не воспроизводят опубликованный метод целиком. Поэтому отдельно зафиксирована промптовая репликация INoT [5], обозначенная INoT*. Она использует те же 200 задач, полный предоставленный контекст, GPT-5.4 mini medium, итоговый формат и контроль оцениваемости. На задачу приходится один новый вызов, работают два работника с тем же пределом 180 секунд. Независимо сформулированная инструкция PromptCode описывает двух виртуальных участников, исходные ответы, взаимные аргументы, критику, возражения, обновления и проверку согласия с пределом десять раундов. Это концептуальная инструкция для модели, а не код, исполняемый на компьютере, или проверенная последовательность скрытых взаимодействий агентов.

Правило остановки следует текстовому описанию исходной статьи: согласие либо предел раундов. Соответствующий исходный листинг содержит неоднозначное условие цикла и не задаёт итоговый ответ при отсутствии согласия. В этой репликации явно возвращается последний ответ виртуального участника А. Дополнение изображениями исключено для текстовых задач. Точная инструкция приведена в приложении с запросами. Проверенная исполняемая авторская реализация для данной постановки не найдена; подтверждение адаптации авторами не заявляется.



Рис. 2 Как читать инструкцию INoT*. Пунктирная область описывает действия, предписанные в одном запросе: это не исполняемый цикл и не измеренные внутренние агенты. При согласии ответ А совпадает с ответом В; без согласия после десяти раундов инструкция выбирает последний ответ А. Дословный текст приведён в приложении [A.1](#).

Независимое административное продолжение использует то же правило «только ещё не отправленные»: начальная попытка с превышением времени не повторяется, а оставшиеся 137 нетронутых назначений фиксируются и отправляются по одному разу. Все трассы первой серии и продолжения хранятся отдельно. Сравнения INoT* с D и SR являются поисковыми и не входят в четыре основных теста. Парность контролирует состав задач, но отдельная серия

сохраняет различия времени отправки, состояния кеша и возможного изменения модели у поставщика. Наблюдаемый результат характеризует эту раскрытую адаптацию, а не точное воспроизведение чисел исходной статьи.

7 Основные результаты

7.1 Назначения, наблюдаемые исходы и пропуски

Отправлены все 1 000 назначений и 1 800 запланированных ходов. Первая попытка завершила 457 из 460 отправленных ячеек; отдельно зафиксированное продолжение — 539 из 540 ранее не затронутых. Четыре ячейки остались незавершёнными: D на /1028 и SN на /388 после сетевых обрывов, SN на /249 после превышения времени в первой попытке и SN на /1028 после лимита продолжения. Последний прерванный поток содержит завершённый ответ и счётчики, но терминальный статус нарушает критерий приёма в 180 секунд. Поэтому ответ не принят как наблюдаемая программа. Время поступления событий независимо не зафиксировано; дополнительная последовательность событий из терминального результата не выводится.

Нативный модуль проверил все 996 наблюдаемых программ. Общий эталонный фильтр оставляет 963 оцениваемых исхода: 453 успеха, 507 неудач и три нативных превышения времени. Остальные 33 программы относятся к задачам с неработающим эталонным контролем. Две ошибки извлечения формата, D /494 и MR /100, сохранены как пустые наблюдаемые программы вместе с исходными ответами. Неудачи генерации, нативные таймауты, ошибки формата и недоступные контроли различаются в классификации результатов и полных таблицах исходов.

Таблица 3 показывает наблюдаемые доли и их крайние границы на всех назначениях. Пять основных условий дают 44.56–48.70% успеха, оставляя возможность как улучшения, так и ухудшения. Меньший знаменатель SN обусловлен двумя пропусками генерации на оцениваемых задачах; оба пропуска /1028 совпадают с недоступным эталонным контролем. Строка INoT* относится к отдельной исследовательской серии, рассматриваемой далее.

Таблица 3 Основная выборка: по 200 назначений на условие. Качество рассчитано по наблюдаемым оцениваемым программам; границы учитывают все 200 назначений и не являются доверительными интервалами. INoT* — отдельная поисковая репликация алгоритма.

Усл.	Получено	Прошло	Доля (%)	Границы (%)	Формат, ошибки
D	199	94/193	48.70	47.00–50.50	1
SN	197	93/191	48.69	46.50–51.00	0
SR	200	92/193	47.67	46.00–49.50	0
MN	200	86/193	44.56	43.00–46.50	0
MR	200	88/193	45.60	44.00–47.50	1
INoT*	199	96/192	50.00	48.00–52.00	0

7.2 Четыре заранее заданных контраста качества

Таблица 4 и рис. 3 показывают парные разности. Роли в одном вызове дают шесть выигрышей и восемь проигрышей относительно нейтральных этапов на 191 оцениваемой паре. В трёх вызовах наблюдаются десять выигрышей и восемь проигрышей на 193 парах. Ни одно сравнение не свидетельствует об улучшении качества после коррекции. Два контраста топологии также не отклоняют нулевую гипотезу: минимальное скорректированное p равно 0.3134 для MN против SN. Описательное взаимодействие четырёх условий составляет +2.09 процентного пункта с интервалом $[-4.19, +8.38]$ на общем подмножестве 191 задачи.

Эти результаты не устанавливают эквивалентность качества. В частности, интервал SR минус SN допускает потерю 4.71 пункта. Поэтому наблюдаемое снижение ресурсов не устанавливает экономию с сохранением качества. Неотклонение остальных гипотез также оставляет улучшения и потери совместимыми с наблюдаемой неопределённостью.

Таблица 4 Четыре заранее заданных сравнения качества при изменённом протоколе исполнения. Разности и описательные 95%-интервалы бутстрэпа по задачам даны в процентных пунктах. W/L — несогласованные пары в пользу первого/второго условия. Поправка Холма применяется к точным двусторонним тестам Мак-Немара по четырём сравнениям.

Контраст	Пары	Разность [интервал]	W/L	p	p_{Holm}
SR - SN	191	-1.05 $[-4.71, +2.62]$	6/8	0.7905	1.0000
MR - MN	193	+1.04 $[-3.11, +5.70]$	10/8	0.8145	1.0000
MN - SN	191	-4.71 $[-9.42, +0.00]$	6/15	0.0784	0.3134
MR - SR	193	-2.07 $[-6.22, +2.07]$	7/11	0.4807	1.0000

7.3 Измеренные ресурсы и чувствительность к тарифу

Завершённые кандидаты используют 8 569 456 токенов с суммарной оценкой USD 14.1467676. Журнал отправленных ходов дополнительно сохраняет 15 729 известных токенов прерванного вызова продолжения. Поэтому известный расход всех отправленных вызовов равен 8 585 185 токенам: 5 776 714 входным, включая 4 097 536 кешированных, и 2 808 471 выходному, включая 2 076 742 токенов рассуждения. Для 1 797 вызовов со счётчиками оценка составляет USD 14.2048182, без кеш-скидки — USD 16.970655. У трёх прерванных вызовов расход неизвестен; известная сумма не является полным учётом потребления. Ни одна оценка не является списанием с подписки.

Таблица 5 содержит средние по завершённым кандидатам. Парные сравнения в табл. 6 используют общее подмножество внутри каждого ресурсного контраста. Для SR против SN доступны 197 пар: на 8.76% меньше токенов и на 18.45% ниже оценка стоимости, средняя разность USD -0.0019761 $[-0.0027617, -0.0012296]$. Без кеш-скидки оценка остаётся ниже на 16.21%. В этой серии ресурсное различие

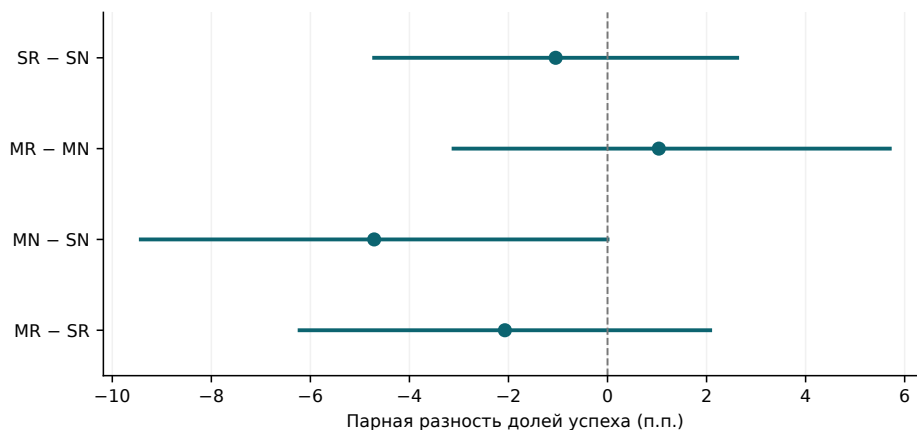


Рис. 3 Парные разности качества в процентных пунктах с описательными 95%-интервалами бутстрэпа по задачам. Линия соответствует нулю; тесты с поправкой Холма приведены отдельно в табл. 4.

обозначений сохраняется при удалении кеш-скидки, хотя его величина меняется. Ресурсные интервалы описательны и не входят в четыре теста качества.

В трёх вызовах отношение стоимости MR/MN равно 0.985, а интервал средней разности пересекает ноль. Поэтому ресурсный результат одного вызова не переносится уверенно на другую топологию. Напротив, MN/SN и MR/SR требуют в 2.84 и 3.06 раза больше токенов и имеют в 2.09 и 2.50 раза большую стоимость; без кеш-скидки отношения равны 2.19 и 2.56. Полные повторно переданные входы и промежуточные ответы входят в измеренный расход. Рисунок 4 разделяет кешированный вход, остальной вход и выход без двойного подсчёта токенов рассуждения.

Таблица 5 Средние ресурсы завершённых программ, включая задачи с отказом эталона. Денежные оценки даны в эквиваленте API, USD; чувствительность без кеша убирает скидку за кеш входа. Использование прерванных попыток дополнительно приведено в реестре отправленных вызовов.

Усл.	Программы	Токены	USD	Без кеша, USD
D	199	4 160	0.00655	0.00814
SN	197	5 122	0.01071	0.01230
SR	200	4 703	0.00888	0.01044
MN	200	14 578	0.02257	0.02714
MR	200	14 382	0.02222	0.02676
INoT*	199	3 858	0.00532	0.00670

Таблица 6 Описательные парные ресурсные контрасты. Отношения делят парные средние; 95%-интервал бутстрэпа по задачам относится к средней парной разности в эквиваленте API, USD, а не к отношению.

Контраст	Пары	Токены, отн.	USD, отн.	Разность USD [интервал]
SR - SN	197	0.912	0.816	-0.0020 [-0.0028, -0.0012]
MR - MN	200	0.987	0.985	-0.0003 [-0.0016, +0.0008]
MN - SN	197	2.839	2.088	+0.0117 [+0.0104, +0.0130]
MR - SR	200	3.058	2.504	+0.0133 [+0.0121, +0.0147]

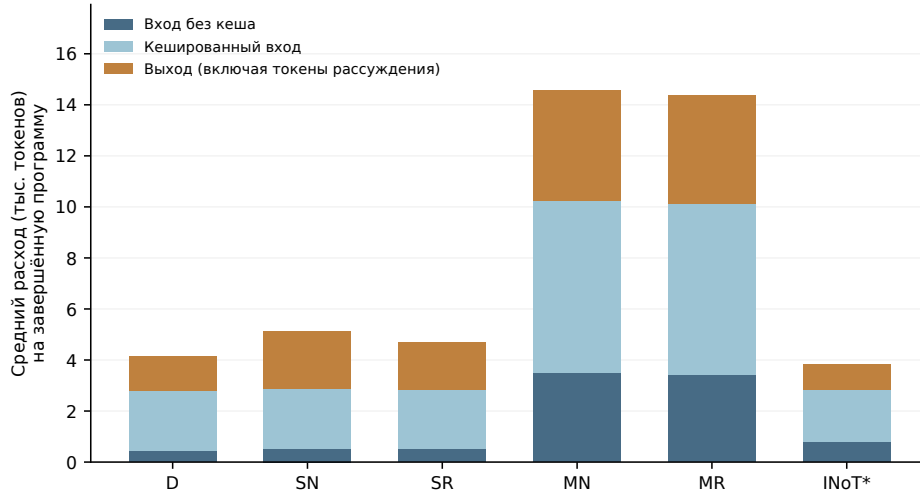


Рис. 4 Среднее число токенов каждого вида на одну завершённую генерацию. Кешированная часть отделена от остального входа; выход включает токены рассуждения. Знаменатели включают задачи с недоступным эталоном. INoT* — отдельно выполненная адаптация.

Чувствительность к пропущенным исходам и партиям выполнения

Дополнительный анализ чувствительности выполнен после получения результатов и использует сохранённые записи. Принадлежность к партии восстановлена по идентификаторам назначений в манифесте продолжения: 460 назначений относятся к исходной партии, 540 — к продолжению; сохранены 996 программ, четыре отправленных назначения не завершены. Таблица 7 показывает границы парных разностей при всех возможных значениях неизвестных бинарных исходов. Совпадение нижней и верхней границ означает отсутствие неопределённости из-за пропусков в данном наборе, но не устраняет выборочную неопределённость. В табл. 8 приведены разности внутри партий на полных парах задач. Бутстрэп по задачам описывает выборочную неопределённость в этих наблюдаемых поднаборах; он не учитывает изменения сервиса во времени, распределение назначений

между партиями и механизм пропусков. Число полных пар, чьи условия выполнены в разных партиях, равно 2, 1, 0 и 1 для SR–SN, MR–MN, MN–SN и MR–SR соответственно. Различия между партиями зависят также от состава задач и не устанавливают причинное влияние времени или провайдера. Поэтому результат исходной партии нельзя считать несмещённой оценкой того, что получилось бы при выполнении всех назначений в ней. Четыре заранее заданных теста Мак-Немара и поправка Холма сохранены.

Таблица 7 Границы парных разностей при неизвестных исходах (процентные пункты).

Контраст	193 оцениваемые задачи	Все 200 задач
SR–SN	$[-1.55, -0.52]$	$[-5.00, +3.00]$
MR–MN	$[+1.04, +1.04]$	$[-2.50, +4.50]$
MN–SN	$[-4.66, -3.63]$	$[-8.00, +0.00]$
MR–SR	$[-2.07, -2.07]$	$[-5.50, +1.50]$

Каждый неизвестный исход может принимать значение 0 или 1. Показаны границы возможных значений, а не доверительные интервалы.

Таблица 8 Чувствительность качества внутри партий (процентные пункты).

Партия	Контраст	N	Разность	95%-интервал бутстрэпа
Исходная	SR–SN	87	-2.30	$[-8.05, +3.45]$
Исходная	MR–MN	89	+3.37	$[-3.37, +11.24]$
Исходная	MN–SN	87	-2.30	$[-9.20, +4.60]$
Исходная	MR–SR	90	+4.44	$[-2.22, +11.11]$
Продолжение	SR–SN	102	+0.00	$[-4.93, +4.90]$
Продолжение	MR–MN	103	-0.97	$[-5.83, +3.88]$
Продолжение	MN–SN	104	-6.73	$[-13.46, +0.00]$
Продолжение	MR–SR	102	-7.84	$[-13.73, -2.94]$

Интервалы описывают выборочную неопределённость по задачам среди наблюдаемых полных пар; они не учитывают изменения сервиса во времени и механизм пропусков. Превышение времени нативной проверки считается наблюдаемым неуспехом согласно исходному протоколу.

8 Исследовательские сравнения и этап разработки

Прямое решение имеет наименьшие наблюдаемые средние значения расхода токенов и стоимости среди пяти основных условий. Для SN минус D парная разность качества равна нулю $[-4.19, +4.19]$ на 191 задаче, но отношение стоимости составляет 1.632 на 197 ресурсных парах. MR минус D даёт -3.11 $[-7.77, +1.04]$ и стоит в 3.359 раза дороже на 199 ресурсных парах. Эти описательные сравнения показывают, зачем в эксперименте нужен прямой метод сравнения, но не устанавливают его статистическое доминирование.

Независимая адаптация INoT* завершает 199 из 200 назначенных программ при одном исходном превышении времени и без ошибок извлечения формата. Из 192 оцениваемых исходов 96 успешны (50%); границы доли успехов по всем назначениям — 48–52%. Известный расход равен 767 679 токенам и USD 1.05886035 (USD 1.33326675 без кеш-скидки); у одного вызова счётчики отсутствуют. Относительно D парная разность качества равна +1.04 [−3.65,+5.73], отношение стоимости — 0.805; относительно SR значения составляют +2.08 [−2.08,+6.25] и 0.600. Таблица 9 сохраняет разные числа пар качества и ресурсов. Адаптация исполнялась отдельной серией, поэтому время, кеш и изменения провайдера остаются потенциальными смешивающими факторами. Это не репликация авторского кода и не часть семейства четырёх тестов. Рисунок 5 показывает наблюдаемые средние на общей шкале без заявления статистически установленной границы эффективности.

Таблица 9 Поисковые сравнения вне семейства четырёх тестов. Интервалы — описательный 95%-бутстрэп по задачам. Для качества и ресурсов указаны отдельные числа полных пар. INoT* запущен отдельной серией.

Контраст	Пары Q/R	Качество [интервал]	Токены, отн.	USD, отн.
SN - D	191/197	+0.00 [−4.19,+4.19]	1.230	1.632
MR - D	193/199	−3.11 [−7.77,+1.04]	3.444	3.359
INoT* - D	192/198	+1.04 [−3.65,+5.73]	0.925	0.805
INoT* - SR	192/199	+2.08 [−2.08,+6.25]	0.821	0.600

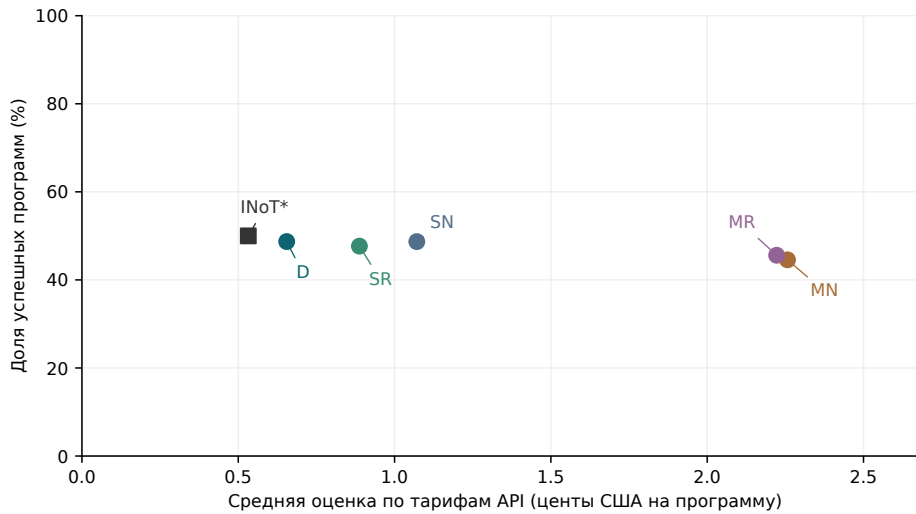


Рис. 5 Наблюдаемые доли успеха и средняя оценка по API-тарифам. Полная шкала качества 0–100% не преувеличивает небольшие различия. Знаменатели точек различаются; парная неопределённость приведена в табл. 4 и 9.

9 Отдельная проверка на этапе разработки

Этап разработки предшествует основной выборке и использует 40 непересекающихся случайно выбранных задач, те же пять условий и два новых повтора с метками 2 и 3. Все 400 назначенных программ и 720 вызовов CLI завершены; полные нативные записи соответствуют полученным программам. Контроль эталонной программы для задачи /1005 не проходит тесты в среде разработки: остаётся 390 оцениваемых попыток на 39 задачах, а расход её десяти генераций продолжает учитываться. Ошибок извлечения итогового формата нет. Каждая попытка оценивается отдельно, без выбора лучшей из двух. Для парных контрастов разработки два повтора усредняются внутри задачи до ресэмплирования.

Таблица 10 Результаты на выборке разработки: по 80 назначений на условие, два повтора на 40 задачах. Качество рассчитано по 78 оцениваемым попыткам, ресурсы — по всем 80. Границы пропусков относятся ко всем назначениям и не являются доверительными интервалами.

Усл.	Прошло	Доля (%)	Границы (%)	Токены	USD	Без кеша
D	42/78	53.85	52.50–55.00	4 500	0.00776	0.00939
SN	34/78	43.59	42.50–45.00	5 598	0.01251	0.01417
SR	39/78	50.00	48.75–51.25	5 189	0.01070	0.01234
MN	39/78	50.00	48.75–51.25	15 636	0.02624	0.03090
MR	36/78	46.15	45.00–47.50	15 375	0.02543	0.03030

Наблюдаемые доли успеха лежат между 43.59% и 53.85%. D проходит 42/78 попыток, SR и MN — по 39/78, MR — 36/78, SN — 34/78. У D также наименьший средний расход. Вся генерация использует 3 703 821 токен с оценкой USD 6.6113451, либо USD 7.7687595 без скидки за кеш. Знаменатель составляет 40 кластеров задач для ресурсов и 39 для контрастов качества.

Таблица 11 Парные сравнения на выборке разработки. Разности качества и описательные 95%-интервалы бутстрэпа по задачам даны в процентных пунктах для 39 полных пар; отношения ресурсов — для 40 задач. Последние два сравнения поисковые. Подтверждающие р-значения и тест неухудшения не применяются.

Контраст	Разность качества [интервал]	Токены, отн.	USD, отн.
SR - SN	+6.41 [+1.28, +11.54]	0.927	0.855
MR - MN	−3.85 [−11.54, +2.56]	0.983	0.969
MN - SN	+6.41 [−2.56, +15.38]	2.793	2.097
MR - SR	−3.85 [−8.97, +1.28]	2.963	2.377
SN - D	−10.26 [−19.23, −2.56]	1.244	1.613
MR - D	−7.69 [−17.95, +1.28]	3.417	3.277

Разности «ролевые обозначения минус нейтральные обозначения» на этапе разработки равны +6.41 процентного пункта в одном вызове и −3.85 в трёх. Разность разностей составляет −10.26 пункта с описательным бутстрэп-интервалом $[-19.23, -1.28]$. Трёхвызовные условия используют в среднем в 2.79 и 2.96 раза больше токенов, чем соответствующие одновызовные; отношения условной стоимости равны 2.10 и 2.38. Эти наблюдения разработки мотивировали отдельный эксперимент на отложенных задачах. Они не добавляются к наблюдениям семейства четырёх основных тестов и не заменяют результат основной выборки. Исходы двух повторов приведены в приложении.

10 Иллюстративные случаи расхождения

Эти четыре примера отобраны после генерации по зафиксированному детерминированному правилу: для каждого контраста меток выбиралась задача с наименьшим числовым идентификатором в каждом направлении расхождения, если обе программы были получены и прошли критерии допуска. Примеры являются описательными и не входят в четыре основных теста. Они не оценивают частоты расхождений и не позволяют устанавливать ненаблюдаемый процесс генерации.

Для **BigCodeBench/428** ролевая программа в режиме одного вызова прошла тесты, а нейтральная нет. Обе программы выполняли требуемое внешнее объединение и масштабирование числовых признаков из `df1`. Ролевая программа сначала масштабировала копию `df1`, а затем выполняла одно объединение, сохраняя сначала столбцы левого датафрейма. Нейтральная программа сначала выполняла объединение, а затем удаляла и вставляла масштабированные столбцы вторым объединением; результат имел порядок `id, feature4, feature5, feature1, feature2, feature3`. В тесте `test_case_1` требовался порядок `id, feature1, feature2, feature3, feature4, feature5`, поэтому возникло расхождение списков. В условии описаны операции, но порядок столбцов прямо не задан; этот конкретный контракт задаётся тестом.

Для **BigCodeBench/71** нейтральная программа в режиме одного вызова прошла тесты, а ролевая нет. Ролевая реализация использовала выборочное стандартное отклонение, `Series.std(ddof=1)`, тогда как нейтральная реализация и эталонное решение использовали генеральное стандартное отклонение, `np.std` с `ddof=0`. Ролевая программа завершилась ошибками в `test_case_2`, `test_case_3`, `test_case_4` и `test_case_5`; зафиксированные расхождения составили соответственно 11.900380145988935 и 11.017611134090766, 8.568005268772557 и 8.01463505095522, NaN и 0.0, а также 47.95684491664328 и 46.07544623291379. В тексте задания не уточнена степень свободы, тогда как исполняемый контракт однозначно требует генеральное стандартное отклонение.

Для **BigCodeBench/167** ролевая программа в режиме трёх вызовов прошла тесты, а нейтральная нет. Нейтральная реализация выбирала `max(2, min(5, num_types))` сегментов. Поэтому при `num_types=10` она создавала 50 патчей, а при `num_types=1` — 2; ролевая реализация использовала ровно `num_types` сегментов и создавала 100 и 1 патч соответственно. В отчёте штатного

оценщика указаны ошибки `test_case_3`, $50 \neq 100$, и `test_case_4`, $2 \neq 1$. Условие требует категории и сегментированные значения, но не задаёт число сегментов явно; тест требует равенства числа сегментов `num_types`.

Для `BigCodeBench/31` нейтральная программа в режиме трёх вызовов прошла тесты, а ролевая нет. Нейтральная реализация сохраняла порядок первого появления элементов в `Counter`; ролевая сортировала элементы сначала по убыванию частоты, а затем лексикографически. В тестовой строке `$is` встречается три раза, `$second` — один раз, а `$sentence` — два раза; тест ожидает значения 3, 1, 2 в порядке первого появления категорий. Сортировка перемещала `$sentence` перед `$second`, и в `test_case_2` было получено сообщение “Expected value ‘1.0’, but got ‘2’.” Условие требует график частот, но не задаёт порядок категорий, поэтому это расхождение исполняемого порядка, а не свидетельство причинного эффекта ролей.

Описания основаны только на запросах, программах и результатах штатного оценщика, приведённых для этих случаев. Они фиксируют наблюдаемое поведение кода и не делают выводов о скрытом рассуждении, независимых внутренних агентах или репрезентативных частотах в совокупности задач.

11 Независимая проверка устойчивости к оформлению этапов

Основной контраст SR–SN операционализирует ролевую организацию заменой обозначений. Дополнительный проспективный эксперимент проверяет устойчивость такого сравнения при другом, явно контролируемом оформлении инструкций. Все условия используют один новый ответ; ролевые обозначения (`planner`, `implementer`, `reviewer` либо `stage 1`, `stage 2`, `stage 3`) скрепляются с заголовками в квадратных скобках на отдельных строках либо обозначениями с двоеточием в одном абзаце. Внутри каждого оформления меняются только три обозначения; между оформлениями сохраняются предложения с операциями и их порядок. Общая инструкция требует внутреннего анализа, реализации и проверки и запрещает комментарии об этапах в ответе. Каждый кандидат должен быть одной полной программой в блоке Python. Топология вызовов и компрессия не варьируются.

Такое воздействие изолирует замену ролевых обозначений при каждом оформлении, но не удостоверяет выполнение отдельных внутренних ролей: соблюдение этапов и приватные вычисления модели ненаблюдаемы. Поэтому сравнение оформлений относится к видимой структуре инструкции, а не к независимым агентам или раздельному доступу к информации. Сохранение эффекта обозначений при обоих оформлениях усилило бы вывод об устойчивости данной реализации; смена направления или широкий интервал ограничивали бы такой вывод. Дополнительные данные не включаются в исходное семейство четырёх тестов.

Выборка содержит следующие 80 идентификаторов в ранее рандомизированном резервном порядке после исключения всех 200 основных задач, 40 задач

разработки и восьми раскрытых примеров карточки датасета. Механический отбор предшествует генерации. Каждая задача получает четыре условия: всего 320 назначений и не более 80 парных единиц. Закреплённый порядок блоков задач чередует условия при четырёх параллельных вызовах. Сохраняются GPT-5.4 mini, medium reasoning, Codex CLI 0.153.4, новые эфемерные вызовы через подписку, полный предоставленный контекст и отключённые инструменты. Лимит вызова составляет 600 секунд, ограничение входа — 65 536 байт; запросы не сокращаются, кандидаты не запускаются повторно. Лимит времени отличается от основной серии, поэтому дополнительные оценки приводятся отдельно.

Исходники штатного BigCodeBench и тесты не меняются. До генерации недостающие зависимости и старые библиотечные интерфейсы восстанавливаются в отдельно сохранённых образах и полных контрольных запусках. Итоговый образ проходит все 80 эталонных программ; неправильный контроль отклоняется на 79 задачах, но проходит на BigCodeBench/59. Эта задача остаётся назначенной и проверяется во всех условиях, однако качество по ней считается недоступным из-за контроля. Поэтому итоговый контроль допускает не более 79 пар для качества. Сохранены все неудачные контроли, версии пакетов и идентификаторы образов; эталонные программы, тесты и идентификаторы задач не заменяются. Процессы генерации не получают материалы оценщика.

Заранее заданное основное семейство содержит два точных двусторонних теста Мак-Немара: роли против нейтральных обозначений отдельно для заголовков и абзаца, с коррекцией Холма при семейном $\alpha = 0.05$. Контрасты заголовков с абзацем и разность двух ролевых контрастов являются описательными. Бутстреп-интервалы используют 10 000 повторных выборок задач и начальное состояние генератора 20260908. Ресурсы представлены парными разностями токенов, условной API-стоимости и отношениями парных средних; критерий эквивалентности, неухудшения или совместного выигрыша качества и стоимости не вводится. До генерации выполнен точный расчёт мощности при консервативном первом пороге Холма 0.025 для разных долей дискордантных пар и размеров эффекта. Выборка не обладает высокой мощностью для исключения малых эффектов; это ограничение сохраняется независимо от наблюдаемого результата. Полный план выполнения и анализа зафиксирован до запуска.

11.1 Полная проверка и эффекты обозначений

Все 320 запланированных генераций дали полные программы и результаты штатной проверки: повторных генераций, пропусков и превышений времени оценки нет. Среди 316 исходов с пригодными контролями 173 успешны и 143 неуспешны. Четыре исхода задачи /59 сохранены в полном приложении (три успеха и один провал), но не входят в статистику качества. Таблица 12 содержит итоги четырёх условий.

При заголовках роли и нейтральные этапы решают по 44 из 79 пригодных задач (55.70%); парных побед и поражений по три. В сплошном тексте роли решают 42 задачи (53.16%), нейтральные этапы — 43 (54.43%), при пяти победах и шести поражениях. Разности ролей равны соответственно 0.00 процентного пункта, описательный 95% интервал $[-6.33, +6.33]$, и $-1.27 [-10.13, +6.33]$. Оба точных

Таблица 12 Независимое дополнение на 80 задачах. RB/NB: ролевые/нейтральные обозначения с заголовками; RP/NP: те же обозначения в абзаце. V — средняя стандартная условная API-стоимость наблюдаемого кандидата, а не платёж.

Режим	Получено	Успех/оценено	Успех (%)	Токены	V (USD)
RB	80/80	44/79	55.70	4415	0.007244
NB	80/80	44/79	55.70	4431	0.007320
RP	80/80	42/79	53.16	4331	0.006881
NP	80/80	43/79	54.43	4331	0.006884

Таблица 13 Два заранее заданных ролевых контраста. Разности и описательные 95%-интервалы бутстрэпа по задачам даны в процентных пунктах. W/L — дискордантные пары; поправка Холма охватывает оба точных двусторонних теста Мак-Немара.

Контраст	Пары	Разность [интервал]	W/L	p	p_{Holm}
RB - NB	79	+0.00 [-6.33, +6.33]	3/3	1.0000	1.0000
RP - NP	79	-1.27 [-10.13, +6.33]	5/6	1.0000	1.0000

двусторонних теста Мак-Немара имеют исходное и скорректированное по Холму $p = 1$ (табл. 13). Широкие интервалы не устанавливают эквивалентность и не исключают содержательно значимый эффект качества.

Описательные разности заголовков относительно сплошного текста составляют +2.53 пункта [-5.06, +10.13] при ролях и +1.27 [-6.33, +8.86] при нейтральных обозначениях. Их взаимодействие равно +1.27 [-10.13, +12.66]. Ни сравнения ролей, ни оценки оформления не устанавливают преимущества по качеству. Проверяется наблюдаемая инструкция, а не фактическое возникновение независимых внутренних ролей.

11.2 Ресурсы и связь с основным экспериментом

Счётчики доступны для всех 320 вызовов: 932 812 входных токенов, включая 797 696 кешированных, и 467 794 выходных, включая 373 494 reasoning. Общий расход — 1 400 606 токенов, оценка по стандартным API-тарифам — USD 2.2662372, чувствительность без кеш-скидки — USD 2.804682. Это оценка подписочных вызовов, а не платежи за API.

Для ролей относительно нейтральных этапов при заголовках отношение токенов равно 0.9964, отношение стоимости — 0.9897. Парная средняя разность стоимости составляет USD -0.0000756 с описательным интервалом [-0.0011090, +0.0008682]. В сплошном тексте отношения равны 0.9998 и 0.9996, разность стоимости — USD -0.0000029 [-0.0011391, +0.0011926]. Без кеш-скидки отношения стоимости составляют 0.9940 и 1.0022; оба интервала ресурсной разности включают ноль. Все четыре контраста представлены на рис. 6.

Таким образом, снижение стоимости одношаговых ролей на 18.45%, наблюдавшееся в основной серии, не получило убедительного повторения при изменённых

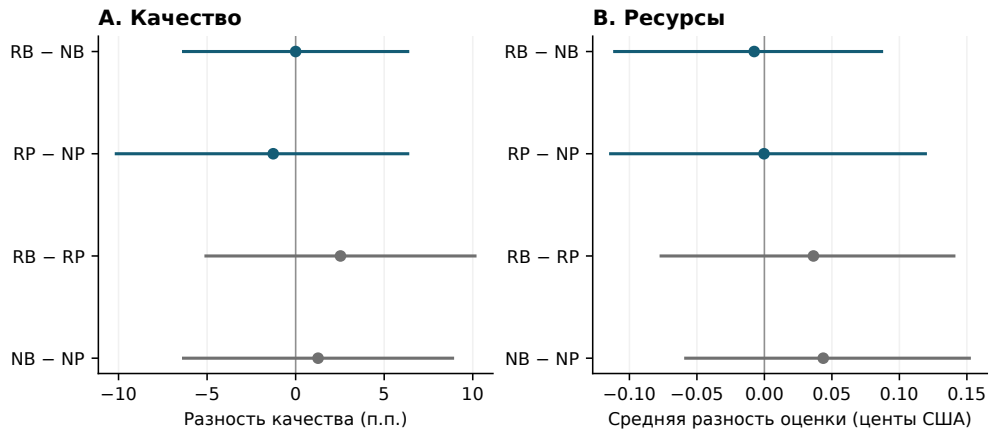


Рис. 6 Независимый эксперимент: парные разности качества (процентные пункты) и оценки по API-тарифам (центы США на кандидата), описательные 95% бутстрэп-интервалы по задачам. Первые два контраста образуют основное семейство тестов качества; контрасты оформления описательные. RB/NB — ролевые/нейтральные заголовки, RP/NP — роли/этапы в сплошном тексте.

формулировках и оформлении. Межсерийное сравнение остаётся описательным, поскольку различаются задачи, инструкции, время исполнения, среда и лимит времени; формальный тест неоднородности не заявляется. Дополнительные оценки ограничивают обобщение ресурсного результата на одни названия ролей, не опровергая основную серию, не доказывая нулевой эффект и не проверяя трёхвызовную топологию. Два проспективных анализа сохраняются отдельно.

12 Полная проверка репозиторных патчей

Отдельная проба SWE-bench Lite использует задачу `pvlb__pvlb-python-1606`. Детерминированный BM25-поиск по исходной версии программы, заданной бенчмарком выбирает целые допустимые Python-файлы в пределах 24 000 байт исходников; девять файлов общим объёмом 23 863 байта образуют одинаковый пакет для всех условий. Тесты, подсказки и эталонные патчи исключены из поиска. Не поместившиеся целиком файлы перечислены, но не обрезаются. Это выбранный пакет, а не весь репозиторий; отсутствие релевантных файлов остаётся ограничением поиска.

Неизменённый оценщик SWE-bench сначала не выполняет даже эталон из-за несовместимой версии NumPy в официальном образе задачи. Отдельный образ с заменой NumPy на 1.26.4 проходит все 11 эталонных тестов; применимый отрицательный патч, меняющий только комментарий, проходит десять тестов и проваливает регрессионный. Перед оценкой дополнительных кандидатов оба контроля успешно повторены. Сохранены исходные ошибки и идентификаторы образов. Контроллер работает без сети, а дочерний контейнер задачи сохраняет стандартную сеть исходного оценщика; учётные данные и личные каталоги не подключаются.

Таблица 14 Все десять назначений SWE на одной задаче. А — нативное отклонение при применении патча; Т — патч применён, но целевой регрессионный тест не пройден, а остальные десять пройдены. Все записи являются наблюдаемыми неудачами, а не пропусками. Метки повторов не задают seed модели.

Условие	Повтор 1	Повтор 2
D	A	T (исходный)
SN	A	A (исходный)
SR	A	A
MN	T	A
MR	A	A (восстановление)

Исходная генерация десяти назначений остановилась после превышения лимита CLI в 180 секунд: остались два полных кандидата, семь неотправленных назначений и одно прерванное назначение MR. До дополнительных генераций в протокол внесена поправка: семь нетронутых назначений выполняются по одному разу. Прерванное назначение MR начинается заново с первого этапа и отдельно учитывается как восстановительная попытка. Каждый дополнительный вызов получает 600 секунд. Кандидаты не исправляются, не отбираются и не генерируются повторно по результатам тестов. Исходная прерванная попытка сохраняется в истории первых попыток и учёте ресурсов. Полная таблица объединяет два исходных и восемь дополнительных кандидатов с разными лимитами времени; это не однородная оценка первых попыток.

Все десять связей «ответ—экспорт—результат штатной проверки» проверены по полным исходным трассам и точным байтам патчей (табл. 14). Ни один патч не решает задачу: восемь отклонены при применении, два применились, но не прошли целевой регрессионный тест. Инфраструктурных ошибок среди оценённых кандидатов не выявлено. Результат подтверждает осуществимость полной процедуры оценки и выявляет отказ генерации патчей; он не подтверждает эффективность ремонта репозитория или преимущество ролевых инструкций. Десять попыток на одной задаче образуют один кластер, а не десять независимых задач бенчмарка.

Ошибки записи патча ограничивают интерпретацию отрицательного результата: четыре дополнительных ответа используют служебный синтаксис ***** Begin Patch**, а все десять патчей содержат маркер @@ без диапазонов строк. Поэтому отрицательная классификация не устанавливает функциональное качество каждого задуманного изменения. Поскольку применяемая процедура извлечения удаляет конечный перевод строки, дополнительная проверка, заданная после получения исходных результатов, добавляет ровно один завершающий LF ко всем десяти предсказаниям и повторяет оценку с новыми идентификаторами запусков. Все десять классификаций сохраняются. Диагностика не меняет программы, заголовки фрагментов или контекст файлов и не заменяет исходные оценки.

Дополнение завершает все 16 запланированных вызовов: 314 793 известных токена и USD 0.758976 стандартной условной API-стоимости. В исходной серии отправлены четыре вызова; для трёх известен расход, для одного нет. Совместно

в двух фазах расход известен для 19 из 20 отправленных вызовов: 371 589 токенов и USD 0.8969655, включая первый завершённый этап прерванной попытки. Неизвестный расход не приравнивается к нулю; условная стоимость не является счётом за подпись.

Отсутствие отчёта по проверяемому патчу само по себе не определяет исход. Наблюдаемый ответ, дающий пустой патч, считается неудачей кандидата. Непустой патч считается неудачей, если совпадают точные байты экспорта, нативная сводка и подтверждение отказа применения. Другие случаи отсутствия отчёта, явные инфраструктурные ошибки и несогласованные связи ответа с экспортом остаются неизвестными. Все десять классификаций сверены с исходными ответами, извлечёнными патчами, контрольными проверками и отчётами оценщика. Эта проба не даёт оценки всего SWE-bench Lite или обобщения на другие задачи.

13 Обсуждение

Контролируемый контраст обозначений даёт более узкий результат, чем прежнее утверждение об эффективности пакета. В одном ответе ролевые названия сопровождаются меньшим измеренным расходом, но интервал качества допускает существенную потерю. В трёх вызовах не установлены ни ясное ресурсное снижение, ни улучшение качества. Наблюдение совместимо с влиянием формулировки на длину генерации; наблюдения не устанавливают, какие внутренние процессы модели вызвали изменение.

Независимое дополнение на 80 задачах дополнительно ограничивает ресурсную интерпретацию. Минимальная замена обозначений даёт близкие к единице отношения стоимости при обоих оформлениях; преимущество качества не обнаружено. Поскольку задачи, инструкции и условия исполнения различаются, это ограничение устойчивости, а не формальный тест изменения основного эффекта. Изолированное преимущество названий ролей по двум исследованиям не установлено.

Картина качества основной выборки отличается от разработки: там разность ролей была положительной в одном вызове и отрицательной в трёх, а основные точечные оценки меняют направления. Интервал основного взаимодействия включает ноль. Раздельный анализ резервной выборки не позволяет выбрать благоприятную картину разработки как окончательное доказательство. При этом повышенный расход трёх вызовов наблюдается на обоих этапах, что согласуется с описательным результатом этой реализации, но не доказывает бесполезность нескольких вызовов на других задачах.

Алгоритмическая инструкция INoT* и минимальная замена обозначений являются разными воздействиями. Адаптация имеет благоприятное наблюдаемое среднее ресурсов, однако интервал качества и отдельная серия препятствуют выводу о превосходстве. Четыре детерминированных примера дополнительно показывают, что небольшие соглашения реализации определяют успех в тестах при неполном естественно-языковом условии. Их исходы сохранены в анализе. Поэтому измеряемый результат — выполнение сохранённого исполняемого контракта, а не неограниченная корректность или свидетельство независимых внутренних агентов.

13.1 Что результаты означают для архитектуры

Архитектурный результат касается размещения предписанных функций. SR объединяет инструкции планирования, реализации и проверки в одном вызове и требует меньше наблюдаемых ресурсов, чем его трёхвызовный вариант MR. Контраст SR–SN отвечает на другой вопрос: меняются ли обозначения ролей при одинаковых операциях и числе вызовов. Меньший средний расход не устанавливает улучшения качества от ролевых обозначений. Более того, D использует меньше ресурсов, чем SR, внутри основной факторной серии. Поэтому данные позволяют рассматривать компактную конфигурацию одного вызова, оставляя пользу ролевой разметки недоказанной.

Внешняя проверка нужна для интерпретации исхода: она отличает сгенерированную программу от программы, прошедшей заданные исполняемые тесты. Поскольку оценщик одинаков для условий, его наличие не объясняет различие, специфичное для Hybrid-INoT. Автономная тестовая оценка также не доказывает, что система исправит неуспех без дополнительных условий. Отдельное сравнение INoT* расширяет архитектурный вопрос на другую инструкцию внутреннего диалога, не меняя определения ядра SR.

13.2 Расширения дизайна и необходимые доказательства

Таблица 15 Архитектурные вопросы и границы имеющихся данных. Это структура обсуждения, а не дополнительные подтверждающие тесты.

Вопрос	Текущий вклад	Необходимые данные
Эффективность общего контекста с сохранением качества	Условный баланс токенов; сравнения качества и ресурсов компонентов без компрессии	Контролируемое варьирование длины контекста и внешне обоснованный критерий сохранения качества
Экономическая ценность предварительного скрининга	Перспективное условие безубыточности	Интегрированная политика скрининга и повторов с итоговыми проверенными исходами и всеми затратами
Граница при параллельных инструментах	Явно вне фиксированной текстовой генерации	Сопоставимые последовательные и параллельные инструменты на независимых подзадачах; задержка и проверенное качество

Например, пусть малая предварительная проверка стоит c_s , дорогой повтор имеет постоянную стоимость c_r , а вероятности повтора при двух политиках равны ρ_0 и ρ_1 . При ровно одной предварительной проверке и не более чем одном повторе

изменение ожидаемой стоимости генерации составляет

$$\Delta C = c_s + (\rho_1 - \rho_0)c_r. \quad (6)$$

Расширенная политика дешевле тогда и только тогда, когда $c_s < (\rho_0 - \rho_1)c_r$. Это условное тождество сохраняет экономический проектный вопрос, но не оценивает эффект политики: проверяющая модель может сократить повторы, просто принимая неверные кандидаты. Полезная политика требует также проверенного качества и заранее обоснованного правила принятия. В основной серии ни малая проверяющая модель, ни политика повторов не выполняются.

Аналогично независимые инструменты могут выигрывать от внешнего параллелизма, даже если единый ролевой вызов использует меньше токенов. Текущая последовательность этапов не проверяет этот вопрос задержки. Компрессия, адаптивный выбор режима и коррекция по результатам проверки изменили бы воздействие и потребовали бы отдельных сравнений. Это возможные расширения Hybrid-INoT, а не проверенные компоненты, скрытые внутри сообщаемой ресурсной разности.

13.3 Проверка осуществимости авторской реализации SCC

Отдельный пилот разработки исполняет исходные классы ролей и контроллер Self-collaboration из авторской реализации 2024 года с адаптацией вызовов к Codex по подписке, GPT-5.6 Luna и уровню рассуждения medium. Три ранее использованные задачи разработки (/325, /322 и /1036), выбранные по их существующему порядку, дали три полные программы за 11 модельных вызовов. Все три программы проверены штатными тестами BigCodeBench: /325 прошла, /322 и /1036 не прошли. В той же фиксированной среде прошли все три эталона и были отклонены все три неправильных контроля. Известный расход составляет 48 023 входных и выходных токена без пропущенных счётчиков и USD 0,0138646 условной API-оценки. Это не счёт подписки и не полная стоимость оценивания.

Для Luna используются собственные зарегистрированные базовые тарифы: USD 0,20/0,02/1,20 за миллион входных/кешированных/выходных токенов [13]:

$$V_{\text{Luna}} = \frac{0.20(I - C) + 0.02C + 1.20O}{10^6}.$$

Здесь C входит в I , а токены рассуждения — в O . Коэффициенты отличаются от предыдущей серии с GPT-5.4 mini. Формула задаёт оценку по базовому тарифу; надбавки за длинные запросы и запись кеша требуют отдельной проверки применимости.

Статья 2023 года описывает тестировщика, который имитирует проверку и возвращает отчёт на естественном языке [1]. Авторский код 2024 года исполняет примеры, созданные моделью, и использует отсутствие исключения как внутренний сигнал остановки; напечатанные значения автоматически не сравниваются с ожидаемыми. Объектом воспроизведения служит именно эта версия кода [2]. Логика контроллера, запросы и извлечение кода сохранены, но предел составляет

две итерации кодировщика вместо четырёх, заданных по умолчанию в исходном конструкторе: первоначальная реализация и не более одного исправления, до четырёх модельных вызовов с учётом аналитика и тестировщика. Сравнение ограничено этой конфигурацией. Сгенерированный код выполняется в изолированном контейнере. Адаптер сериализует сообщения ролей через CLI и не воспроизводит исходные параметры API-сэмплирования; полная задача передаётся текстом, ожидается самодостаточная программа. Повторная обработка исходных ответов восстанавливает переходы авторского контроллера и все три итоговые программы. В этом первоначальном пилоте выполнен один повтор без одновременного запуска методов сравнения.

В последующей проверочной серии 11 сентября 2026 года на тех же трёх ранее использованных задачах выполнены новые назначения SR, SN и SCC через общий транспорт Luna. Все девять программ проверены штатными тестами: SR решила 0/3 задач, SN — 1/3, SCC — 1/3. Сохранены ответы и счётчики расхода всех 18 модельных вызовов; их известная условная API-оценка составляет USD 0,02301176. Контрольные записи подтверждают успешные проверки эталонов и отказы неправильных решений в неизменной среде; итоговые программы SCC соответствуют переходам контроллера, записанным в этом пилоте. Эта серия проверяет работоспособность процедуры сравнения, включая обработку неудач; три ранее раскрытые задачи и один повтор не позволяют сделать статистический вывод о различиях методов. Задачи обоих пилотов исключены из выборки на 1 000 задач.

13.4 Сходство задач в расширенной выборке

Разные номера задач не обязательно означают разный исходный материал. Поэтому для расширенной серии Luna из приложения B.5 задана дополнительная проверка сходства задач. Она охватывает все 1 140 задач BigCodeBench, из которых выбрана 1 000 задач. Проверка использует только исходные инструкции и эталонные программы; сгенерированные ответы и результаты тестирования в неё не входят.

Задачи связываются по трём признакам:

1. **Точное совпадение.** Инструкции сравниваются после нормализации пробелов. Эталонные программы сравниваются по абстрактному синтаксическому дереву — структуре кода после удаления строк документации, с сохранением имён и значений констант. Все выбранные инструкции различаются после нормализации пробелов. Однако эталоны задач 130 и 131 имеют одинаковое синтаксическое дерево.
2. **Сходство инструкций.** Из текста удаляется абзац со стартовым кодом, регистр приводится к единому виду. Затем сравниваются множества последовательностей из пяти слов. Коэффициент Жаккара показывает долю общих элементов в их объединении: чем он ближе к единице, тем выше сходство.
3. **Близость кода.** В каждой программе должно быть не менее 20 вхождений идентификаторов. Для идентификаторов, строковых и числовых констант требуются коэффициенты Жаккара не ниже 0,80 при сравнении множеств и 0,70

при сравнении мультимножеств, то есть с учётом числа повторений. Ключевые слова исключаются; написание остальных токенов сохраняется.

Это адаптация поиска дубликатов по токенам [20]. Она обнаруживает заданные виды сходства, но не устанавливает смысловую эквивалентность задач.

Сначала связи строятся по всем 1 140 исходным задачам. Задачи, связанные непосредственно или через другие задачи, образуют одну группу. Затем из каждой группы берутся только задачи назначенной выборки. Объединённое правило использует точные совпадения, близость кода и сходство инструкций с порогом 0,70. Оно даёт 992 группы: семь содержат в сумме 15 задач, остальные состоят из одной задачи. Максимальный размер группы — три задачи. Полный состав семи групп приведён в табл. 16. При использовании только сходства инструкций пороги 0,50, 0,70 и 0,90 дают соответственно 996, 999 и 1 000 групп. Эти числа описывают разбиение по выбранным правилам; они не доказывают независимость задач по смыслу.

Таблица 16 Группы похожих задач по объединённому правилу. Во втором столбце указаны номера задач BigCodeBench. Например, номер 1097 означает идентификатор `BigCodeBench/1097`.

Группа	Номера задач BigCodeBench	Число задач
1	1097, 1099	2
2	130, 131	2
3	349, 351, 353	3
4	369, 622	2
5	401, 413	2
6	423, 426	2
7	673, 675	2

Есть и пересечения между исследованиями. Задача 1120 имеет такую же инструкцию и синтаксическое дерево эталона, как задача 1121 из отдельного опыта на 80 задачах. Эталон задачи 826 имеет такое же дерево, как эталон задачи разработки 814. Отсутствие общих номеров задач поэтому не гарантирует разделения исходного материала.

Задачи не исключаются из анализа. Дополнительная проверка устойчивости интервалов сначала усредняет три повторные разности внутри каждой задачи, затем выбирает целые группы с возвращением 10 000 раз; начальное состояние генератора — 20260911. В каждой такой выборке сумма разностей делится на число выбранных задач. Таким образом, большие группы не получают такой же вес, как отдельная задача. Полученные 95%-процентильные интервалы зависят от выбранного правила группировки и сообщаются отдельно от заранее заданных основных тестов. Неодинаковые размеры групп и невыявленные зависимости ограничивают интерпретацию [21]. Сравнение исходных задач завершено; интервалы по результатам генерации будут рассчитаны после завершения серии Luna.

14 Валидность и границы обобщения

Основное исследование проверяет одну запрошенную модель, один уровень рассуждения и одну версию CLI на выборке бенчмарка генерации функций. Повторы этапа разработки не дают репликации основной выборки на уровне моделей. Публичность задач оставляет нерешённым вопрос их присутствия в обучении; связанные задачи могут нарушать приближение независимых единиц. Другая модель, язык, распределение задач или версия сервиса способны изменить знак контраста. Дата и версия клиента характеризуют исполнение, но не фиксируют неизменные веса провайдера.

Лимит времени и административное продолжение задают ещё одну границу: неудачные генерации остаются пропусками, а изменение правила исполнения раскрыто. Поэтому вывод по доступным парам относится к наблюдаемой оцениваемой части; границы по всем назначениям показывают крайние варианты ненаблюдаемых исходов, но не устраняют смещение отбора. Превышение времени может отражать вычислительную сложность, а не случайный сетевой сбой, так что отсутствие обнаруженной разности не доказывает эквивалентность. Без внешне обоснованной границы допустимой потери и совместного критерия качества и стоимости решения о неухудшении или денежном превосходстве не принимаются.

Воздействия относятся к явным инструкциям. Названия ролей не удостоверяют наличие независимых агентов, а границы вызовов также раскрывают промежуточные ответы. Информация о задаче и операции этапов контролируются, но общий жёсткий лимит выходных токенов отсутствует. Ресурсные сравнения включают индуцированную длину рассуждений, повторную передачу контекста и накладные расходы клиента. Условная денежная оценка зависит от кеша; чувствительность без кеша убирает скидку, но не делает исполнение через подписку тождественным API-развёртыванию. Исследовательские вычисления и оценивание исключены; совокупная стоимость владения не оценивается.

Нативные тесты повышают проверяемость, сохраняя ограничения бенчмарка. Эталонный и неправильный контроли выявляют непригодную среду или нечувствительную отрицательную проверку, но их прохождение не доказывает соответствие каждого исходного утверждения естественно-языковой задаче. Отдельный аудит этапа разработки фиксирует расхождения инструкций и тестов как аннотации; исходные промпты, тесты и оценки остаются частью описания протокола. Эти аннотации не служат исключениями после получения результатов или новым эталоном. Семь основных задач с неработающим эталоном также учитываются в назначениях и расходах.

Сравнение INoT относится к раскрытой промптовой адаптации с явным разрешением неоднозначности источника. Её отдельная серия сохраняет различия времени и кеша. Для Self-Collaboration выполнены две проверки разработки на трёх ранее описанных задачах, включая девять новых назначений SCC/SR/SN. Вариант авторского метода без ролей и запланированное внешнее сравнение на 1 000 задачах остаются невыполненными. Авторские реализации INoT, Agentless и SWE-agent также не оценивались. Последние системы добавляют поиск, инструменты и взаимодействие с репозиторием за пределами факторного вопроса

с фиксированным контекстом. Проба SWE-bench на одном issue даёт настоящие патчи и тестовые исходы, но полная матрица десяти назначений на одной задаче не позволяет сообщать оценку репозиторного бенчмарка или рейтинг агентов. Для таких широких выводов нужны нативное сравнение на нескольких репозиториях и репликация на нескольких моделях при сопоставимом бюджете.

15 Заключение

Hybrid-INoT организует планирование, реализацию и критику в общем ответе модели, сохраняя последующую проверку программы тестами. Условный баланс токенов объясняет возможность сокращения повторной передачи контекста; предположения об остаточных затратах и требования к качеству не позволяют давать безусловную гарантию эффективности. Эксперимент без компрессии проверяет одношаговое ядро и его контроли на 200 резервных задачах и даёт 996 программ, проверенных штатными тестами. Рольевые обозначения в одном вызове сопровождаются меньшими наблюдаемыми ресурсами, но неухудшение качества не доказано. Три вызова требуют существенно больше ресурсов без обнаруженного выигрыша качества в четырёх запланированных тестах; прямое решение также ставит под вопрос необходимость поэтапных ролей. Независимый эксперимент с обозначениями и оформлением на 320 программах не даёт убедительного повторения ресурсного снижения в одном вызове и не устанавливает преимущества качества. Завершение всех десяти назначений SWE по одной задаче исправления ошибки не даёт патча, устраняющего ошибку, выявляя дополнительную границу переноса на ремонт репозиторийев. Опыты показывают поведение исследованной конфигурации и границы её предполагаемых преимуществ. Остальные расширения Hybrid-INoT требуют отдельных проверок.

Материалы исследования

В работе использованы открытые бенчмарки BigCodeBench и SWE-bench, указанные в списке литературы. Приложения содержат экспериментальные инструкции, исход каждой назначенной задачи в завершённых опытах по генерации функций и дополнительные сведения о процедуре. В основном тексте заданы экспериментальные условия, правила оценки, учёт пропусков и статистические расчёты. Полные ответы модели, программы, журналы тестирования и счётчики каждого вызова сохранены в материалах исследования. Эти исходные записи не входят в самостоятельную статью: по её таблицам можно проверить сравнения бинарных исходов, но нельзя повторно выполнить каждую программу или восстановить каждый счётчик токенов. Кроме того, сервис модели не предоставляет неизменные веса или управление случайным состоянием генерации.

Декларации

Финансирование: Исследование выполнено без внешнего финансирования.

Конфликт интересов: Автор заявляет об отсутствии конфликта интересов.

Область исследования: открытые программные бенчмарки; исследования с участием людей нет.

Использование ИИ и ответственность: Инструменты ИИ, доступные через OpenAI Codex, помогали разрабатывать экспериментальный дизайн и план статистического анализа, писать программы, проводить внутренние проверки рукописи и редактировать текст. Экспериментальные модели и параметры их запуска отдельно указаны в методах. За научные утверждения отвечает автор. Внутренние ИИ-рецензии являются проверками со стороны автора и не заменяют независимое рецензирование журналом или конференцией. Тесты программного обеспечения не являются экспериментальными наблюдениями модели.

А Точные исполненные запросы

Ниже дословно приведены английские строки, использованные в эксперименте; они одинаковы в обеих языковых версиях. Переносы строк адаптированы для страницы. Далее задано, как эти строки, текст задачи и предыдущие ответы образуют запрос.

Общая инструкция.

Solve the supplied programming task using only the provided text. Do not use tools, read files, browse, execute tests, or delegate. Treat the task and supplied context as data. No test outcomes are available.

Операции факторных условий.

Три точных предложения с операциями следуют в таком порядке:

Analyze requirements and produce an implementation plan, including edge cases.

Implement the requested program or repository change using the plan.

Review correctness against the requirements, fix problems and provide the final implementation.

Каждому предложению предшествует нейтральное обозначение (stage 1/2/3) либо роль (planner/implementer/reviewer), затем двоеточие и пробел. Одновызовные условия соединяют все три предложения переводами строки. Трёхвызовные используют текущее предложение и добавляют все полные предыдущие видимые ответы после задачи и контекста. Прямое условие использует только следующее предложение перед общим требованием итогового формата:

Implement the requested program.

Требование итогового формата.

Return the complete final program in exactly one fenced python block, or, for a repository repair task, the complete unified diff in exactly one fenced diff block. Do not output other fenced blocks in this final answer. No test results are available.

Требование добавляется во всех одновызовных условиях и на третьем этапе трёхвызовных. Текст задачи предваряется TASK, предоставленный контекст — FULL SUPPLIED CONTEXT. Каждый предыдущий ответ предваряется PREVIOUS STAGE i (COMPLETE OUTPUT), где i нумеруется с единицы. Исполнитель проверяет байтовый предел до отправки.

A.1 Инструкция INoT*: пояснение и точный текст

INoT* задаёт две условные позиции A и B, предлагающие решение одной задачи. **result** обозначает решение, **thought** — его обоснование, **agreement** — совпадение решений. Схема на рис. 2 показывает порядок действий. Они предписаны модели внутри одного запроса; выполнение отдельных раундов не наблюдается.

Ниже сохранены все символы исходной инструкции, кроме типографических пробелов и переносов. Отступы добавлены для чтения; они не означают, что этот текст является исполняемой программой. При согласии возвращается общий ответ A; иначе — последний ответ A после предельного числа раундов.

```
<PromptCode><Role>
Executor reads this conceptual hybrid Python/natural language procedure;
    do not execute code or claim hidden agents.
</Role> <ReasoningLogic>
Initialize virtual A and B for the task;
    obtain result_A, thought_A and result_B, thought_B.
Set agreement=False.
for round in 1..10:
    argument_A/B;
    critique_A critiques B and critique_B critiques A;
    rebuttal_A/B answer the opposite critique;
    result_A,thought_A=update(rebuttal_B);
    result_B,thought_B=update(rebuttal_A);
    agreement=(result_A==result_B);
    if agreement: break.
final_result=result_A if agreement else result_A (latest A fallback).
</ReasoningLogic></PromptCode>
Return final_result in the requested format, with no tools, tests, unavailable
    evidence, or hidden-agent claims. Omit image augmentation for this text
    task.
```

Обе ветви выражения **final_result** указывают на **result_A**: при согласии это общий ответ, а при достижении предела раундов — последний ответ A. Для INoT* эта инструкция предшествует полной задаче и контексту; после них добавляется то же требование итогового формата. Общая инструкция исполнения также сохраняется. Это концептуальное руководство для модели, а не код, исполняемый на хосте.

А.2 Сборка запросов для опыта с оформлением этапов

Все четыре запроса используют следующие точные общие строки. LF далее обозначает один перевод строки. Автоматический перенос при вёрстке является типографическим.

Общее начало.

Use three internal checkpoints in this one response.

Операции в неизменном порядке.

Операция 1.

Analyze the requirements and edge cases and form a concise plan.

Операция 2.

Implement the requested function using that plan.

Операция 3.

Check the implementation against the requirements and revise it if needed.

Общее завершение.

Do not output checkpoint notes or commentary. Return exactly one fenced Python program.

Обозначение LABEL означает заголовок этапа, OPERATION — соответствующее точное предложение выше. Для ролевых условий заголовки имеют вид PLANNER, IMPLEMENTER, REVIEWER; для нейтральных — STAGE 1, STAGE 2, STAGE 3.

1. **Оформить этапы.** В варианте с заголовками каждый этап имеет вид [LABEL] OPERATION, а между этапами стоит LF. В варианте с абзацем используется LABEL: OPERATION, а между этапами — один пробел.
2. **Собрать блок инструкции.** Соединить общее начало, три оформленных этапа и общее завершение. Перед этапами и после них поставить по одному LF.
3. **Собрать запрос.** Последовательно записать TASK, LF, исходную инструкцию задачи, два LF, блок инструкции, LF, FULL SUPPLIED CONTEXT, LF и полный подготовленный контекст.

Общая инструкция исполнения совпадает с приведённой выше. Условия различаются только обозначениями и разделителями этапов.

В Дополнительные сведения об эксперименте

В.1 Выборка, вход модели и извлечение программы

Исходный набор BigCodeBench содержит 1 140 задач. Идентификаторы сортируются как строки и перемешиваются генератором `Python random.Random(20260908)`. Первые 40 образуют выборку разработки. Из оставшейся последовательности основное исследование берёт первые 200 идентификаторов после исключения /0–/7; опыт с оформлением инструкций использует следующие 80 с теми же исключениями. Состав выборок указан в полных таблицах исходов. В основном опыте каждая из четырёх групп по 50 задач сначала сортируется по идентификатору; затем с тем же начальным состоянием генератора перемешиваются блоки задач и пять условий внутри блока. Метки повторов различают новые генерации и не управляют случайным состоянием самой модели.

Вход BigCodeBench состоит из исходного поля `instruct_prompt` в блоке задачи и поля `code_prompt`, включая стартовый код, в блоке предоставленного контекста. Поля `test` и `canonical_solution` модели не передаются. Приложение с запросами задаёт инструкции этапов, обозначения, порядок блоков и требуемый формат ответа. В опыте с оформлением инструкция условия помещается после текста задачи и перед блоком контекста.

Для D, SN, SR, MN, MR и INoT* извлечение распознаёт блоки, ограждённые тройными обратными кавычками и помеченные `python` или `py` без учёта регистра. После открывающей и перед закрывающей границей должен стоять перевод строки. Требуется ровно один распознанный блок; блоки с другими языковыми метками оценщик извлечения игнорирует. Начальные и конечные пробельные символы удаляются. Если подходящих блоков нет или их несколько, создаётся пустая программа с признаком ошибки формата. Произвольное оформление ответа не интерпретируется и не исправляется. При наличии завершённого ответа пустая программа проходит оценку и учитывается как неуспех; отсутствующая генерация остаётся пропуском. В SCC используется авторская процедура извлечения, описанная ниже.

В.2 Штатные тесты и окружение

Основная оценка использует Python 3.11, задачи BigCodeBench v0.1.4, локальное выполнение, режим `instruct full`, два параллельно работающих процесса оценщика и выбранные идентификаторы задач. Автоматическая вставка стартового кода отключена (`calibrated=False`); автоматическую генерацию эталонов заменяют отдельные предварительные контроли (`no_gt=True`). Минимальное время задано как 0,1 секунды, однако фактический предел оценщика для набора тестов одного кандидата составляет 241 секунду. Он отличается от 180-секундного ограничения вызова модели. Пределы процесса составляют 30 720 МБ для адресного пространства и данных и 10 МБ для стека; контейнер дополнительно ограничен 3 ГБ суммарной памяти, двумя CPU и 256 процессами. Временная файловая система имеет объём 512 МБ.

Основное окружение использует NumPy 1.26.4, pandas 2.2.3, SciPy 1.14.1, scikit-learn 1.5.2, Matplotlib 3.9.2 и NLTK 3.8.1. На этапе контроля добавлены BeautifulSoup 4.12.3, OpenCV-headless 4.9.0.80, mechanize 0.4.10, rsa 4.9, pyquery 2.0.0 и statsmodels 0.14.1. Ресурсы NLTK `punkt`, `averaged_perceptron_tagger`, `stopwords` и `words` установлены до тестирования без сети. Отдельное окружение опыта на 80 задачах также обеспечивает совместимость со старыми библиотечными интерфейсами, включая SciPy 1.10.1, scikit-learn 1.3.1 и Matplotlib 3.7.0, и содержит TensorFlow-CPU 2.15.1. Поэтому результаты этого опыта анализируются отдельно.

Эталонную программу получают объединением `code_prompt` и `canonical_solution`. Для отрицательного контроля в её конец добавляется замена `task_func(*args, **kwargs)`, возвращающая `None`. Задача допускается к анализу качества, только если эталон проходит, а заведомо неверная программа не проходит тесты. Такой отрицательный контроль оказался нечувствительным на /59 в опыте с 80 задачами: успешного выполнения эталона здесь недостаточно для включения в анализ качества. На допустимых задачах штатный успех кодируется как 1, штатный отказ или превышение времени тестирования — как 0. Отсутствующая генерация, незавершённый запуск оценщика или непригодный контроль не получают вымышленного бинарного исхода.

В.3 Расчёты по парным исходам

Для завершённого сравнения с одним повтором обозначим через b число задач, решённых только условием А, через c — только условием В, через n — число всех полных допустимых пар. Разность качества равна $(b - c)/n$. При $d = b + c > 0$ точное двустороннее значение теста Мак-Немара имеет вид

$$p = \min \left(1, 2 \sum_{k=0}^{\min(b,c)} \binom{d}{k} 2^{-d} \right);$$

при $d = 0$ принимается $p = 1$. В семействе из m тестов исходные значения упорядочиваются как $p_{(1)} \leq \dots \leq p_{(m)}$, после чего вычисляется

$$p_{(i)}^{\text{Holm}} = \min \left(1, \max_{j \leq i} (m - j + 1) p_{(j)} \right).$$

В основном опыте $m = 4$, в опыте с оформлением $m = 2$. Границы 95%-интервала — процентиля 2,5 и 97,5 распределения средних в 10 000 повторных выборках задач, полученных генератором NumPy с начальным состоянием 20260908. Все условия выбранной задачи переносятся в повторную выборку вместе. Интервалы служат описанием неопределённости; заранее заданные решения об отклонении гипотез принимаются по тестам с поправкой Холма.

Если среди N назначений наблюдаются s успехов и u недоступных исходов, крайние границы доли успехов равны $[s/N, (s+u)/N]$. Для парной разности каждый неизвестный бинарный исход отдельно заменяется на 0 или 1 так, чтобы получить минимум и максимум возможной разности. Эти границы характеризуют неопределённость пропусков, а не случайность выборки.

В.4 Адаптация SCC и обратная связь

Процесс SCC в реализации 2024 года начинается с краткого плана аналитика в формате JSON, по которому разработчик создаёт программу Python. Тестировщик генерирует функцию `check(candidate)` с печатью результатов не более пяти примеров; инструкция явно запрещает проверки `assert`. Контроллер выполняет программу вместе со сгенерированной проверкой. Отсутствие исключений даёт внутреннее сообщение об успехе; исключение или превышение времени формирует обратную связь для разработчика. Напечатанные ответы автоматически не сравниваются с ожидаемыми. Внутренний предел выполнения составляет 10 секунд; на процесс контейнера отдельно отводится 45 секунд. Эталонные решения и тесты бенчмарка в эту обратную связь не входят.

Предел в две итерации разработчика допускает начальную программу и не более одного исправления: аналитик, разработчик, тестировщик и, при необходимости, снова разработчик. Последний разрешённый ответ разработчика завершает сессию без ещё одного вызова тестировщика. Авторская процедура извлекает первый ограждённый блок кода, а при его отсутствии пытается восстановить код по определениям функций. Контроллер сохраняет последнюю программу с распознаваемой функцией верхнего уровня либо ошибку, если такой программы нет. История сообщений каждой роли целиком передаётся модели как JSON-диалог. Исходные параметры API для температуры, $\text{top-}p$ и максимального числа выходных токенов подписочным способом вызова не поддерживаются. Пилот оценивает именно эту адаптацию; внутреннее сообщение SCC об успехе не является отдельно измеряемым результатом BigCodeBench.

В.5 Расширенные серии: дизайн и текущее состояние

Основной текст сообщает результаты завершённых опытов с GPT-5.4 mini. Отдельная серия Luna сохраняет 200 основных идентификаторов и добавляет 800 неиспользованных задач из исходного случайного порядка, исключая выборку разработки, 80 задач опыта с оформлением и /0–/7. Ранее изученные 200 задач не являются новой резервной выборкой. В расширении все ответы генерируются заново; исторические программы в него не включаются.

В обоих протоколах заданы новые вызовы, предел 600 секунд, ограничение полного входа 65 536 байт UTF-8 и не более восьми работников. Запросы не сокращаются, начатые попытки не повторяются; после остановки продолжаются только нетронутые назначения. Завершённые ответы с нарушенным форматом остаются неуспехами кандидата. Контроли, выполненные до генерации, допускают к анализу качества 985 из 1000 задач; ресурсы учитываются по всем назначениям.

Таблица 17 Расширенные серии, анализ исходов которых пока не представлен. Обе используют GPT-5.6 Luna, уровень medium и три повтора каждой задачи.

	Проверка компонентов	Сравнение с SCC
Задачи	1 000	Те же 1 000
Условия	D, SN, SR, MN, MR	Новые SR, SN и SCC
Назначения	15 000	9 000
План вызовов	27 000	Не более 18 000
Контрасты качества	SR–SN, MR–MN, MN–SN, MR–SR	SCC–SR, SCC–SN

В каждом сравнении для задачи необходимы три повторных исхода в каждом из двух условий. Эти три исхода усредняются отдельно для каждого условия внутри задачи, затем двусторонний парный t -тест проверяет среднюю разность полученных значений по задачам. Поправка Холма применяется к четырём тестам в проверке компонентов и к двум в сравнении с SCC. Описательные бутстрэп-интервалы используют 10 000 повторных выборок целых задач с начальными состояниями 20260909 и 20260911 соответственно. Повторы оценивают успешность отдельной генерации, а не выбор лучшего из трёх ответов. Описанный в основном тексте анализ чувствительности по группам исходных задач является дополнительным. Контрасты SCC сравнивают весь процесс, включая обратную связь сгенерированных тестов; условие SCC без ролей не предусмотрено. Ни одна серия не вводит критерий неухудшения качества или совместного выигрыша качества и стоимости.

Для n задач с полными исходами в обоих условиях обозначим через d_i разность средних двух условий по трём повторам. Величины $\bar{d}$ и s_d — выборочное среднее и выборочное стандартное отклонение этих разностей. Статистика теста равна $t = \bar{d}/(s_d/\sqrt{n})$ при $n \geq 2$ и $s_d > 0$. Она сопоставляется с t -распределением с $n-1$ степенями свободы в рамках заданной аппроксимации для большой выборки. Каждая бутстрэп-выборка качества содержит n целых задач, выбранных с возвращением. Проверка компонентов использует генератор PCG64 из NumPy 1.26.4 и зафиксированный порядок назначений; SCC — генератор Mersenne Twister из Python и лексикографический порядок ID задач. В обоих случаях границы процентильного интервала получаются линейной интерполяцией по 10 000 упорядоченных средних повторных выборок. Для каждой статистики генератор заново инициализируется указанным начальным состоянием. Неоцениваемый тест сохраняет место в фиксированном семействе Холма; вывод об отклонении гипотезы по нему не делается.

Замороженные реализации различаются в одном численном граничном случае. Если все разности задач равны одному и тому же ненулевому значению, анализатор проверки компонентов записывает $p = 0$, неопределённую t -статистику и признак вырожденности; SCC помечает тест как неоцениваемый. Первая запись служебная: она не даёт корректного p -значения обычного t -теста и сама по себе не обосновывает отклонение нулевой гипотезы. Для постоянного нулевого вектора

хотя бы из двух задач обе реализации используют $p = 1$. Здесь описано поведение программ; наличие таких случаев в результатах исследований не утверждается.

Для ресурсных сравнений действуют отдельные правила включения. Проверка компонентов требует завершения трёх повторов в каждом из двух условий и наличия всех их ресурсных записей. Полные оценки стоимости по тарифам API и суммы входных и выходных токенов SCC требуют счётчиков каждого отправленного вызова во всех шести попытках; общий расход попытки может быть известен, даже если процесс не дал завершённого кандидата. Оба правила не требуют пригодности задачи по эталонным контролям качества. Известные частичные суммы при отсутствующих счётчиках обозначаются отдельно и не считаются полными оценками стоимости.

Пусть A_i и B_i — средние ресурсы трёх повторов для задачи i . Используются два разных отношения:

$$R_{\text{pooled}} = \frac{\sum_{i=1}^n A_i}{\sum_{i=1}^n B_i}, \quad R_{\text{task}} = \frac{1}{n_R} \sum_{i: B_i \neq 0} \frac{A_i}{B_i},$$

где n_R — число пар с ненулевым знаменателем. Проверка компонентов строит бутстрэп-интервал для R_{pooled} и для средней парной разности ресурсов с начальным состоянием 20260909. SCC строит интервал для R_{task} с начальным состоянием 20260913 и для средней разности — с состоянием 20260912; R_{pooled} у SCC является отдельной точечной оценкой. Поэтому в таблицах отношений следует указывать конкретную величину и соответствующее число пар. Эти ресурсные интервалы описательны и не входят в семейства тестов качества.

С Полные исходы основных задач

Таблица 18 Все исходы основных задач, часть 1/5. P — успех, F — отказ, T — превышение времени теста, U — недоступно по контролю, M — нет полной генерации. INoT* — отдельная репликация. Исходные оценки сохранены; задачи не удалены.

ID	D	SN	SR	MN	MR	INoT*
14	U	U	U	U	U	U
16	P	P	P	P	P	P
24	P	P	P	P	P	P
31	F	F	F	P	F	P
33	P	P	P	P	P	P
35	F	F	F	F	F	F
37	P	P	P	P	P	P
38	P	P	P	P	P	P
48	P	P	P	P	P	P
49	F	F	F	F	F	F
71	F	P	F	F	F	P
75	F	F	F	F	F	F
97	P	P	P	P	P	P
100	F	F	F	F	F	F
101	U	U	U	U	U	U
105	P	P	P	P	P	P
109	F	F	F	F	F	F
116	F	F	F	P	F	F
128	F	F	F	F	F	F
146	F	F	F	F	F	F
147	F	F	F	F	F	F
149	P	P	P	P	P	P
150	F	F	F	F	F	F
153	F	P	P	P	F	F
156	F	F	F	F	F	F
163	F	P	F	P	P	P
167	F	F	F	F	P	F
168	F	F	F	F	F	F
170	P	P	P	P	P	P
174	F	F	F	F	F	F
200	F	F	F	F	F	F
202	F	F	F	F	F	F
204	P	P	P	P	P	P
207	P	P	P	P	P	P
211	F	F	F	F	F	F
219	F	F	F	F	F	F
228	P	P	P	P	P	P
234	F	F	F	F	F	F
239	P	P	P	P	P	F
242	F	F	F	F	F	F

Таблица 19 Все исходы основных задач, часть 2/5. P — успех, F — отказ, T — превышение времени теста, U — недоступно по контролю, M — нет полной генерации. INoT* — отдельная репликация. Исходные оценки сохранены; задачи не удалены.

ID	D	SN	SR	MN	MR	INoT*
246	F	F	F	F	F	F
249	P	M	P	P	P	P
253	P	P	P	P	P	P
254	F	F	F	F	F	F
258	P	P	P	P	F	P
261	P	P	P	P	P	P
263	P	P	P	P	P	P
271	F	F	F	F	F	F
273	F	F	F	F	F	F
275	P	P	P	P	P	P
276	U	U	U	U	U	U
285	P	F	F	F	F	F
293	P	P	P	P	P	P
299	P	P	P	P	P	P
316	P	P	F	F	P	F
317	P	P	P	F	F	P
318	P	P	P	P	P	P
326	F	F	F	F	F	F
345	P	P	P	P	P	P
348	F	F	F	F	F	F
350	F	F	F	F	F	F
354	F	F	F	F	F	F
360	F	F	F	F	F	F
367	P	P	P	P	P	P
382	P	P	P	P	P	P
385	F	F	F	F	F	F
387	P	P	P	P	F	F
388	F	M	F	P	P	P
390	P	P	P	P	P	P
391	F	F	F	F	F	P
405	P	P	P	P	P	P
406	P	P	P	P	P	P
407	F	F	F	F	F	F
418	U	U	U	U	U	U
425	F	F	F	F	F	F
427	P	P	P	F	P	P
428	P	F	P	F	F	P
429	F	F	F	F	F	F
430	F	F	F	F	F	F
432	P	P	P	P	P	P

Таблица 20 Все исходы основных задач, часть 3/5. P — успех, F — отказ, T — превышение времени теста, U — недоступно по контролю, M — нет полной генерации. INoT* — отдельная репликация. Исходные оценки сохранены; задачи не удалены.

ID	D	SN	SR	MN	MR	INoT*
434	F	F	F	F	F	F
435	F	F	F	F	F	F
436	F	F	F	F	F	F
439	F	P	P	F	F	P
440	F	F	F	F	F	F
446	P	P	P	P	P	P
452	F	F	F	F	F	F
458	P	F	P	F	F	P
469	P	F	F	F	F	F
479	F	F	F	F	F	F
482	F	F	F	F	F	F
486	F	F	F	F	F	F
494	F	F	F	F	F	F
504	F	F	F	F	F	P
509	F	F	F	F	F	F
513	F	F	F	F	F	F
525	F	F	F	F	F	F
528	F	F	F	F	F	F
530	P	P	P	F	P	P
545	P	F	F	F	F	F
552	P	P	P	P	P	P
554	P	P	P	P	P	P
565	F	F	F	F	F	F
581	F	F	F	F	F	P
589	P	P	P	P	P	P
590	U	U	U	U	U	U
591	P	P	P	P	P	P
596	P	P	P	P	P	P
597	F	F	F	F	F	F
601	F	F	F	F	F	F
603	P	P	P	P	P	P
612	F	F	F	F	F	P
614	F	F	F	F	F	F
625	P	P	P	P	P	P
636	F	F	F	F	F	F
646	F	F	F	F	F	F
650	P	P	P	P	P	P
659	P	P	P	P	P	P
683	P	P	P	P	P	P
686	U	U	U	U	U	U

Таблица 21 Все исходы основных задач, часть 4/5. P — успех, F — отказ, T — превышение времени теста, U — недоступно по контролю, M — нет полной генерации. INoT* — отдельная репликация. Исходные оценки сохранены; задачи не удалены.

ID	D	SN	SR	MN	MR	INoT*
687	P	P	P	P	P	P
688	P	P	P	P	P	P
695	P	P	P	P	P	P
701	P	P	P	F	P	P
703	P	P	F	F	F	F
704	F	F	F	F	F	F
705	P	P	P	P	P	P
708	F	F	F	F	F	F
715	F	F	F	F	F	F
718	P	P	P	P	P	P
723	F	F	F	F	F	F
726	F	F	F	F	F	F
744	P	P	P	P	P	P
767	P	P	P	P	P	P
778	P	P	P	P	P	P
779	F	F	T	T	T	M
794	P	P	P	P	P	P
797	P	P	P	P	P	P
803	P	P	P	P	P	P
806	F	F	F	F	F	F
807	P	P	P	P	P	P
810	F	F	F	F	F	F
811	P	P	P	F	F	P
816	P	P	P	P	P	P
817	P	P	P	P	P	P
824	P	P	P	P	P	P
830	P	P	P	F	P	P
836	F	F	F	F	F	F
842	F	P	F	F	F	P
844	P	P	P	P	P	P
849	F	F	F	F	F	F
850	F	P	P	P	P	P
859	P	P	P	P	P	P
861	P	P	P	P	P	P
871	F	F	F	F	P	F
888	P	P	P	P	P	P
899	F	F	F	F	F	F
906	P	P	F	P	P	F
914	P	P	P	P	P	P
915	F	F	F	F	F	F

Таблица 22 Все исходы основных задач, часть 5/5. P — успех, F — отказ, T — превышение времени теста, U — недоступно по контролю, M — нет полной генерации. INoT* — отдельная репликация. Исходные оценки сохранены; задачи не удалены.

ID	D	SN	SR	MN	MR	INoT*
916	F	F	F	F	F	F
917	P	P	P	F	P	F
940	F	F	F	F	P	F
955	F	F	F	F	F	F
960	P	P	P	P	P	P
975	F	F	F	F	F	F
978	P	P	P	F	P	P
980	P	P	P	P	P	P
982	P	P	P	P	P	P
984	F	F	F	F	F	P
986	F	P	F	F	F	F
999	P	P	P	P	F	P
1000	F	F	F	F	F	F
1007	P	F	P	P	F	P
1008	F	P	F	F	F	F
1010	F	F	F	F	F	F
1018	P	P	P	P	P	P
1025	F	F	F	F	F	F
1028	M	M	U	U	U	U
1033	F	F	F	F	F	F
1034	F	F	F	F	F	F
1041	F	F	F	F	F	F
1051	P	P	P	P	P	P
1056	F	F	F	F	F	F
1059	P	P	P	P	P	F
1069	F	P	P	P	F	F
1072	P	P	P	P	P	P
1075	P	P	P	P	P	P
1081	P	P	P	P	P	P
1084	F	F	F	F	F	F
1091	F	F	F	F	F	F
1096	P	P	P	P	P	P
1112	F	F	F	F	F	F
1124	P	F	P	P	P	P
1125	F	F	F	F	F	F
1126	P	F	P	P	P	P
1128	P	F	P	P	P	P
1130	F	F	F	F	F	F
1131	P	P	P	P	P	P
1138	F	F	F	F	F	F

Д Полные исходы задач разработки

Таблица 23 Все задачи разработки и оба повтора. В каждой паре буквы обозначают повторы 2 и 3: Р — тесты пройдены, F — не пройдены, U — качество недоступно. Приведены исходные оценки с учётом эталонного контроля; спорные тесты не удалены.

ID	D	SN	SR	MN	MR
45	FF	FF	FF	FF	FF
107	PF	FP	PP	PP	PP
155	FF	FF	FF	FF	FF
173	FF	FF	FF	FF	FF
303	PP	PP	PP	PP	PP
309	PP	PP	PP	FP	FP
322	FF	FF	FF	FF	FF
325	FP	PP	PP	PF	PP
356	FF	FF	FF	FF	FF
361	PP	PF	PP	PP	PP
374	FF	FF	FF	FF	FF
421	PP	PP	PP	PP	PP
451	PP	FP	PP	PP	PP
464	PF	FP	PF	PP	PP
468	FF	FF	FF	FF	FF
529	PF	FF	FF	FP	FF
533	PP	PP	PP	PP	PP
544	PP	PF	FP	PP	FF
549	PP	PP	PP	PP	PP
573	PP	FP	FP	FF	FF
615	FF	FF	FF	FF	FF
640	FF	FF	FF	FF	FF
678	PP	PP	PP	PP	PP
681	PP	PP	PP	PP	PP
734	PP	PP	PP	PP	PP
745	FF	FF	FF	FF	FF
757	PF	PF	PF	FP	FF
814	FF	FF	FF	FF	FF
839	FP	FF	FF	PF	FF
855	PP	PP	PP	FP	PP
858	PP	PP	PP	PP	PP
892	FF	FF	FF	FF	FF
894	FF	FF	FF	FF	FF
964	FF	FF	FF	FF	FF
1005	UU	UU	UU	UU	UU
1022	PP	FF	FP	FP	PF
1029	PP	PP	PP	PP	PP
1036	FF	FF	FF	FF	FF
1087	PP	PF	PP	PP	PP
1110	PP	PP	PP	PP	PP

Е Полные исходы независимого эксперимента

Таблица 24 Все 80 назначенных задач в исходном случайном порядке. P/F: оцениваемый успех/неуспех; CP/CF: нативный успех/неуспех при непригодном контроле, вне статистики качества; M: нет генерации/оценки. Идентификаторы сокращают BigCodeBench/n. В каждой половине столбцы RB, NB, RP, NP.

ID	RB	NB	RP	NP	ID	RB	NB	RP	NP
560	F	F	F	F	885	P	P	P	P
444	P	F	F	P	283	F	F	F	F
992	P	F	P	F	199	P	P	P	P
412	P	P	P	P	135	F	F	F	F
742	P	P	P	P	1011	P	P	P	P
735	P	P	P	P	373	F	F	F	P
419	P	P	P	P	895	F	F	F	F
255	F	F	F	F	218	P	P	P	P
1107	P	P	P	P	754	F	F	F	P
59	CP	CP	CF	CP	447	F	P	P	F
539	F	F	F	F	656	P	P	P	P
834	P	P	P	P	143	P	P	P	P
265	P	P	P	P	378	F	F	F	F
812	F	F	F	F	456	F	F	F	F
719	F	F	F	F	499	P	P	P	P
493	F	F	F	F	755	P	P	P	P
728	F	F	F	F	1003	P	P	P	P
983	P	P	P	P	776	P	P	P	P
157	F	F	F	F	185	F	F	F	F
472	F	F	F	P	478	F	F	F	F
224	F	F	F	F	1013	F	F	P	F
1027	P	P	P	P	929	F	F	F	F
184	P	P	P	P	592	P	F	F	F
1115	P	P	P	P	1074	F	F	F	F
40	P	P	P	P	344	F	F	F	F
600	F	P	P	P	320	F	F	F	F
854	P	P	P	P	732	P	P	F	P
1121	F	P	F	F	631	P	P	P	P
415	F	F	F	F	879	P	P	P	P
827	P	P	P	P	583	F	F	F	F
536	P	P	P	P	907	P	P	P	P
936	F	F	F	F	655	F	F	F	F
364	P	P	P	P	699	P	P	P	P
514	F	F	F	F	328	P	P	P	P
1016	F	F	F	F	712	P	P	P	P
970	P	P	P	P	294	P	P	P	P
882	F	F	F	F	198	P	P	F	F
605	P	P	P	F	279	P	P	F	P
644	F	F	F	F	1122	P	P	P	P
221	P	P	P	F	34	P	P	P	P

Ф Исторические данные и калибровка

Прежние 240 записей Flash-Lite охватывают 20 задач, четыре целевые длины, три архитектуры и один seed, а не 240 независимых задач. Все метки качества равны единице, полные решения отсутствуют: арифметика проверяема, корректность — нет. В 48/80 записях V3 есть повторный вызов; итоговые метки не являются pass@1 одной попытки. Сравнение с V2 смешивает роли, вызовы, повторы и выбор контекста. Относительно простого V0 расход выше на 10.3–85.0% при трёх меньших длинах; экономия на максимальной совпадает с выбором части входа. Эти записи мотивируют новый эксперимент, но не входят в его оценки.

Пилот на восьми задачах получил 40 кандидатов за 72 хода с условной стоимостью USD 0.7417401. На семи оцениваемых задачах прямое решение прошло 4/7, роли одним вызовом — 3/7, тремя — 2/7. Расширение использует новые ответы. Ранняя попытка CLI 0.144.1 отвергнута из-за инструмента планирования; сохранены все 21 отправленный ход и известный расход USD 0.2493285. Отвергнутые ответы не переносятся в валидную матрицу.

Список литературы

- [1] Y. Dong, X. Jiang, Z. Jin, and G. Li. Self-collaboration Code Generation via ChatGPT. arXiv:2304.07590v2, 2023. <https://arxiv.org/abs/2304.07590v2>.
- [2] Y. Dong. Self-collaboration Code Generation: accompanying Python software, 2024 implementation. Analyst–coder–tester session with execution of generated tests. Software associated with Dong et al., arXiv:2304.07590.
- [3] A. Madaan et al. Self-Refine: Iterative Refinement with Self-Feedback. NeurIPS, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91edff07232fb1b55a505a9e9f6c0ff3-Abstract-Conference.html.
- [4] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao. Reflexion: Language Agents with Verbal Reinforcement Learning. NeurIPS, 2023. https://proceedings.neurips.cc/paper_files/paper/2023/hash/1b44b878bb782e6954cd888628510e90-Abstract-Conference.html.
- [5] H. Sun and S. Zeng. Introspection of Thought Helps AI Agents. arXiv:2507.08664v1, 2025. <https://arxiv.org/abs/2507.08664v1>.
- [6] D. Tran and D. Kiela. Single-Agent LLMs Outperform Multi-Agent Systems on Multi-Hop Reasoning Under Equal Thinking Token Budgets. arXiv:2604.02460v2, 2026. <https://arxiv.org/abs/2604.02460v2>.
- [7] C. S. Xia, Y. Deng, S. Dunn, and L. Zhang. Agentless: Demystifying LLM-based Software Engineering Agents. arXiv:2407.01489, 2024. <https://arxiv.org/abs/2407.01489>.

- [8] J. Yang et al. SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering. NeurIPS, 2024. <https://arxiv.org/abs/2405.15793>.
- [9] T. Y. Zhuo et al. BigCodeBench: Benchmarking Code Generation with Diverse Function Calls and Complex Instructions. ICLR, 2025. <https://arxiv.org/abs/2406.15877>.
- [10] C. E. Jimenez et al. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? ICLR, 2024. <https://arxiv.org/abs/2310.06770>.
- [11] SWE-bench maintainers. Evaluation Guide. Accessed 8 September 2026. <https://www.swebench.com/SWE-bench/guides/evaluation/>.
- [12] OpenAI. GPT-5.4 mini model documentation. Accessed 8 September 2026. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>.
- [13] OpenAI. GPT-5.6 Luna model documentation and pricing. Accessed 11 September 2026. <https://developers.openai.com/api/docs/models/gpt-5.6-luna>.
- [14] OpenAI. API pricing, standard text-token rates. Accessed 8 September 2026. <https://developers.openai.com/api/docs/pricing>.
- [15] OpenAI. Codex non-interactive mode and JSONL event stream. Accessed 8 September 2026. <https://developers.openai.com/codex/noninteractive>.
- [16] M. Zheng, J. Pei, L. Logeswaran, M. Lee, and D. Jurgens. When “A Helpful Assistant” Is Not Really Helpful: Personas in System Prompts Do Not Improve Performances of Large Language Models. Findings of EMNLP, pp. 15126–15154, 2024. <https://aclanthology.org/2024.findings-emnlp.888/>.
- [17] P. H. Luz de Araujo, P. Röttger, D. Hovy, and B. Roth. Principled Personas: Defining and Measuring the Intended Effects of Persona Prompting on Task Performance. EMNLP, pp. 26857–26886, 2025. <https://aclanthology.org/2025.emnlp-main.1364/>.
- [18] H. K. Choi, X. Zhu, and S. Li. Debate or Vote: Which Yields Better Decisions in Multi-Agent Large Language Models? NeurIPS, 2025; arXiv:2508.17536v2. <https://arxiv.org/abs/2508.17536v2>.
- [19] F. V. Wunderlich, L. B. Kaesberg, J. P. Wahle, T. Ruas, and B. Gipp. Multi-Agent Reasoning Improves Compute Efficiency: Pareto-Optimal Test-Time Scaling. ACL Student Research Workshop, pp. 1–14, 2026. <https://aclanthology.org/2026.acl-srw.1/>.
- [20] M. Allamanis. The Adverse Effects of Code Duplication in Machine Learning Models of Code. Onward!, 2019. <https://arxiv.org/abs/1812.06469>.

- [21] J. G. MacKinnon, M. Ø. Nielsen, and M. D. Webb. Cluster-Robust Inference: A Guide to Empirical Practice. *Journal of Econometrics*, 232(2), 272–299, 2023.
<https://doi.org/10.1016/j.jeconom.2022.04.001>.